Autour du problème du voyageur de commerce: optimisation de cycles logistiques sous contraintes

Thème: Cycles et boucles

Gilles BAYALA , 10101
Option SII

Source : dreamstime.com



Comment modéliser et optimiser efficacement des cycles logistiques soumis aux contraintes géométriques d’un réseau routier réel ?



Source : pngtree.com

Sommaire

1) Présentation du problème
2) 3 approches
3) Résultats
4) Conclusion

1) Présentation du problème

2) 3 approches

3) Résultats

4) Conclusion

Présentation du problème

Problème du voyageur de commerce*:
NP-difficile

Pas de solution exacte
(n sommets: n ! possibilités)

Alternative: Usage d'heuristiques / de métaheuristiques



Source : chemisky.github.io

*Travelling Salesman Problem

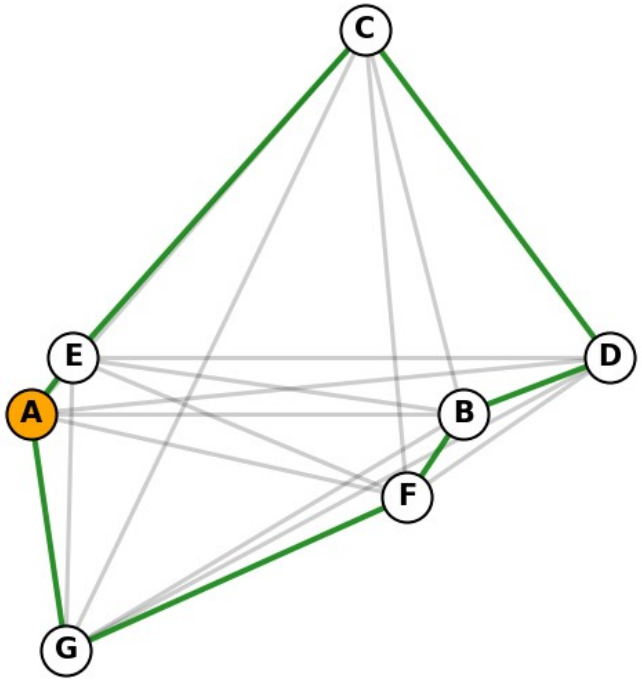
Objectifs

But: Construire des cycles dans un graphe non orienté

A : Sommet de départ

Chemin trouvé est une solution du TSP si:

- Passe au moins 1 fois par chaque sommet via arêtes existantes
- Retourne au sommet de départ



Chemin:
A->E->C->D->B->F->G->A

Exemple de solution au TSP

1) Présentation du problème



2) 3 approches

3) Résultats

4) Conclusion

Algorithme de Colonies de Fourmis

- ACO: Ant Colony Optimization

  (Algorithme de Colonies de Fourmis)

- Marco Dorigo, 2004
- Fonctionne par vagues



Source : exterminatek.ca

Fonctionnement de l’ACO

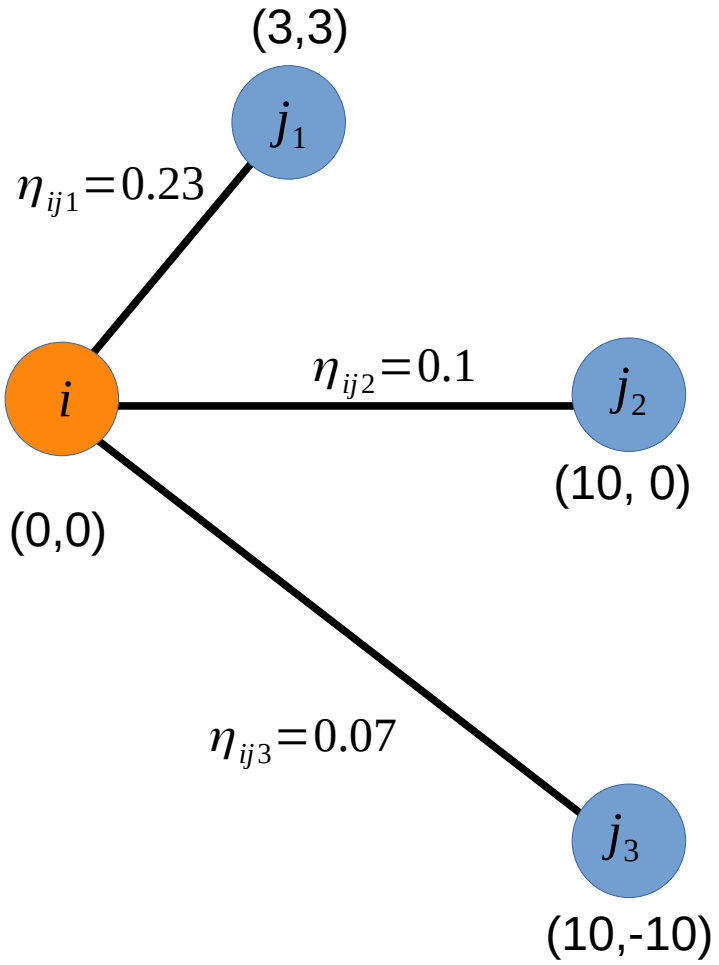
Paramètres mis en jeu

η : *Paramètre de visibilité*

$$\eta_{ij} = \frac{1}{d_{ij}}$$

Avec d_{ij} : *Distance entre sommets* i *et* j

β : *Poids associé à* η $(\beta = 1)$



Paramètre de visibilité sur portion de graphe

Paramètres mis en jeu

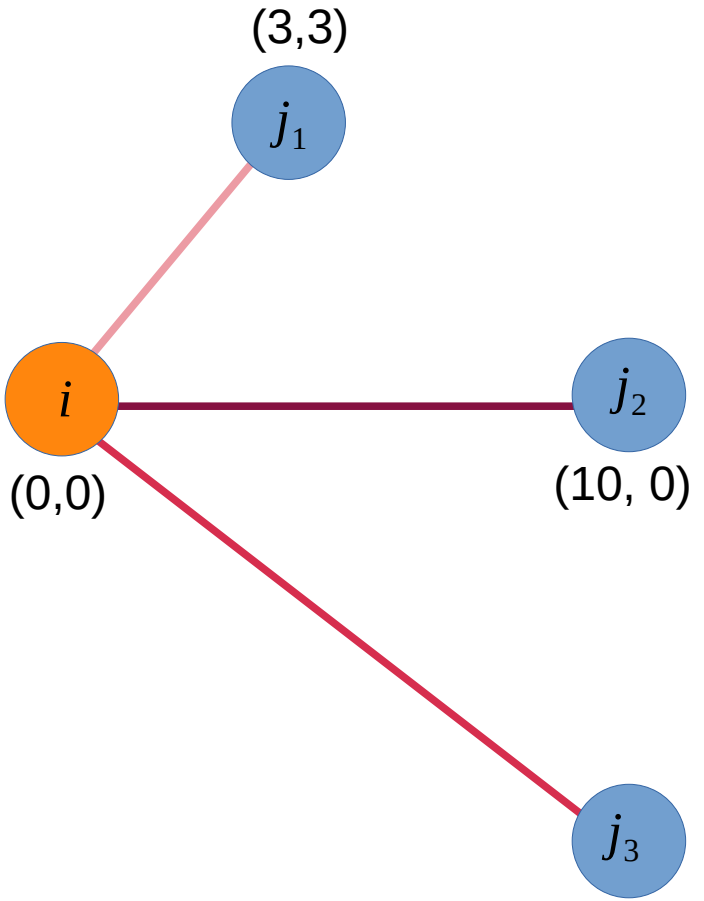
τ : *Paramètre associé à la présence de phéromones*

$$\tau_{ij}(t+1)=(1-\rho)\cdot\tau_{ij}(t)+\sum_{k=1}^{m}\Delta\tau_{ij}^{k}$$

Et $$\Delta\tau_{ij}^{k}=\frac{M}{L_k}$$

Avec
ρ : *Coefficient d'évaporation*
$\tau_{ij}(t)$: *Intensité de la trace de phéromones sur arête ij*
M : *Quantité de phéromones transportée*
L_k : *Longueur totale parcourue par fourmi k*

α : *Poids associé à* τ $(\alpha=1)$



Portion de graphe

Taux de phéromones en dégradé de Magenta

Choix de sommet ?

Choix de sommet probabiliste

Probabilité d'aller du sommet i au sommet j

$$P_{ij} = \frac{(\tau_{ij})^{\alpha} \cdot (\eta_{ij})^{\beta}}{\sum_{k \in \text{sommets autorisés}} (\tau_{ik})^{\alpha} \cdot (\eta_{ik})^{\beta}}$$

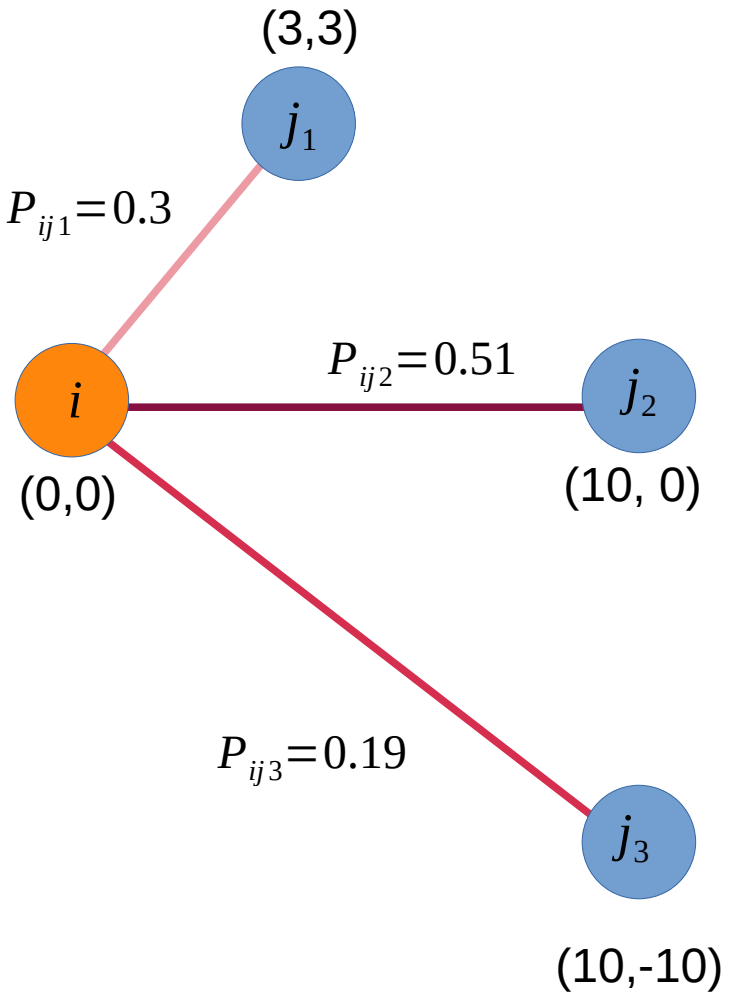
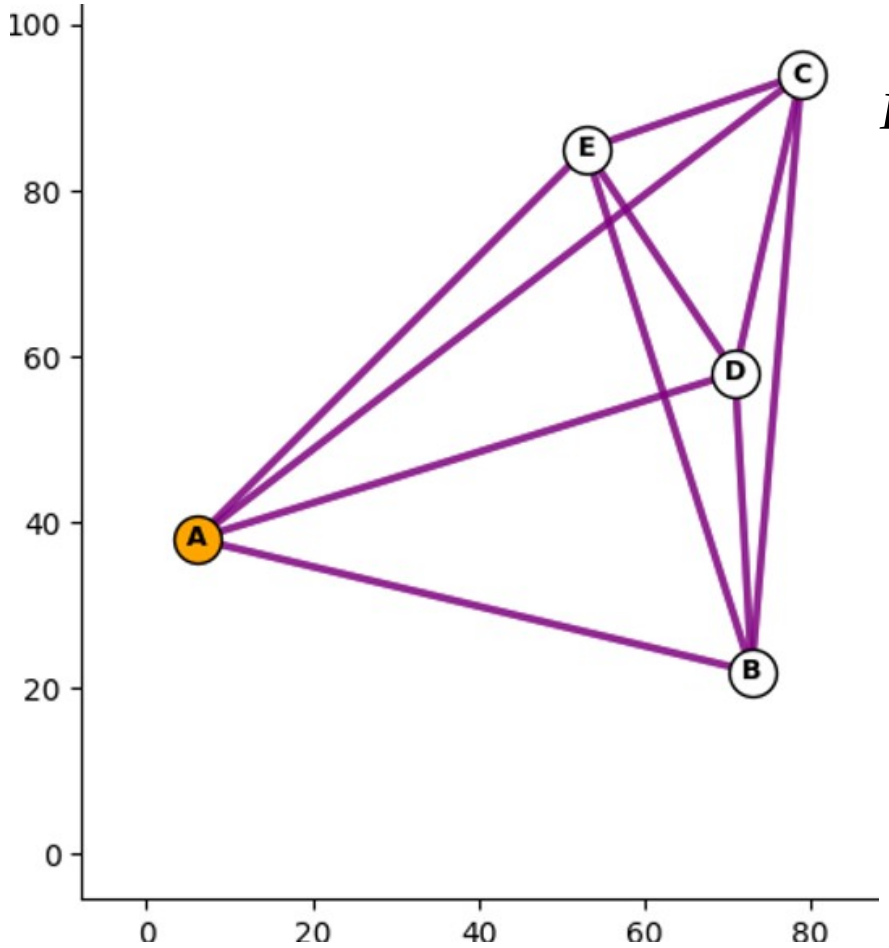


Illustration du choix d'un sommet

Fonctionnement de l’ACO

Vague initiale



Chemins empruntés par les fourmis de la vague initiale

Déplacement dépendant du paramètre de visibilité η

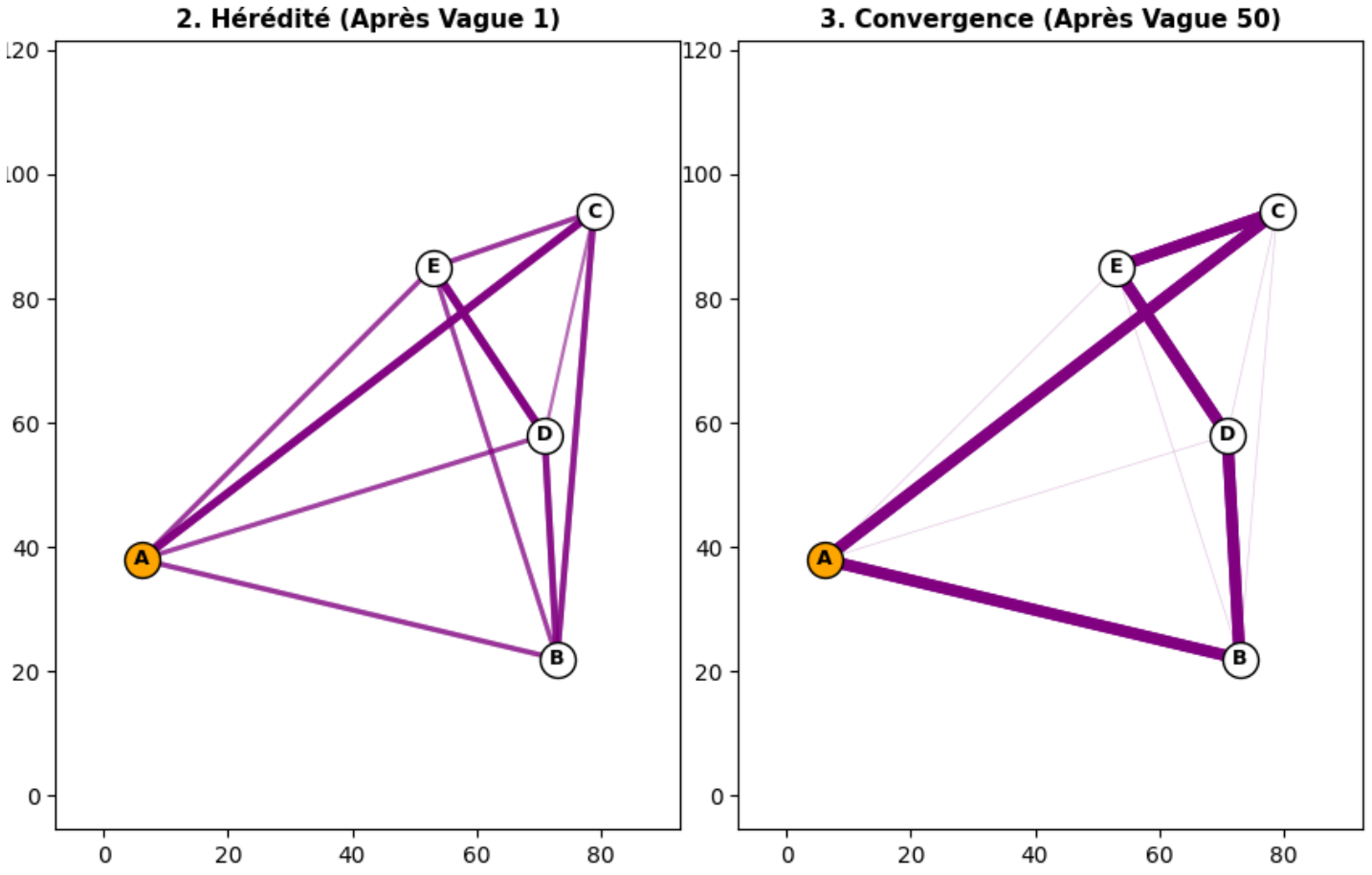
Fourmi 1 :
Chemin : A→ C→ B→ D→ E→ A
Distance parcourue : 342.39

Fourmi 2 :
Chemin : A→ D→ E→ B→ C→ A
Distance parcourue : 331.93

Fourmi 3 :
Chemin : A→ E→ C→ D→ B→ A
Distance parcourue : 256.23

Vagues suivantes

- Évaporation d'une quantité de phéromones (facteur ρ)
- Exploration du graphe en fonction des phéromones (τ) et de la visibilité (η)



Alternative: ACO Dynamique

- Poids associé à la visibilité variable

$$\beta(c) = \beta_{base} + \gamma \cdot \left(\frac{Q-c}{c+0.01}\right)$$

Q : *Nombre total de colis à livrer*
c : *Nombre de colis restants à livrer*

Rappel ACO Classique :
$P_{ij1} = 0.3$
$P_{ij2} = 0.51$
$P_{ij3} = 0.19$

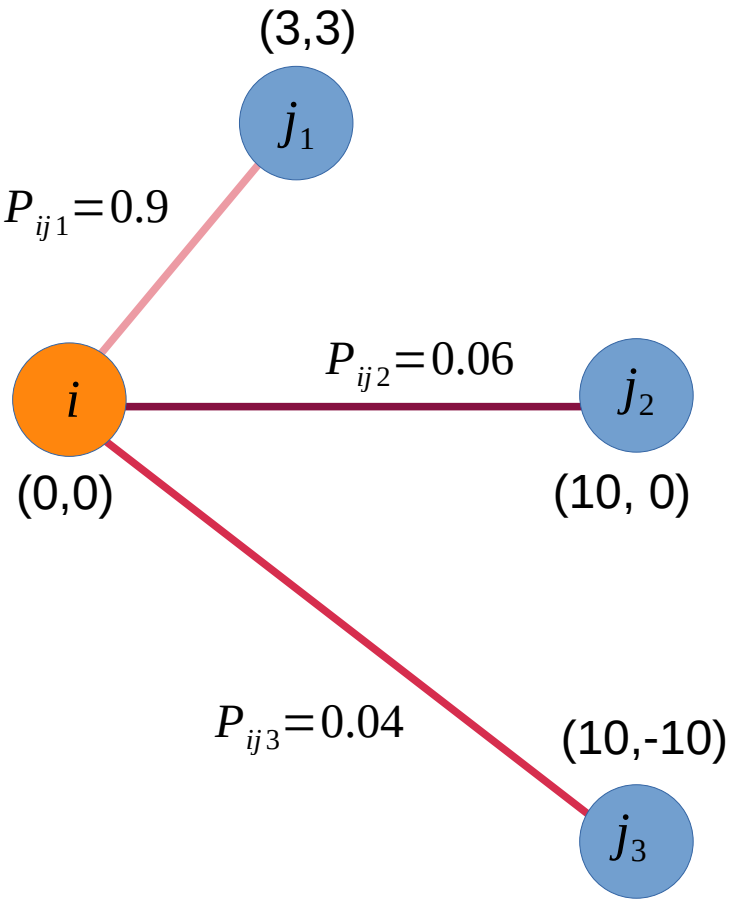
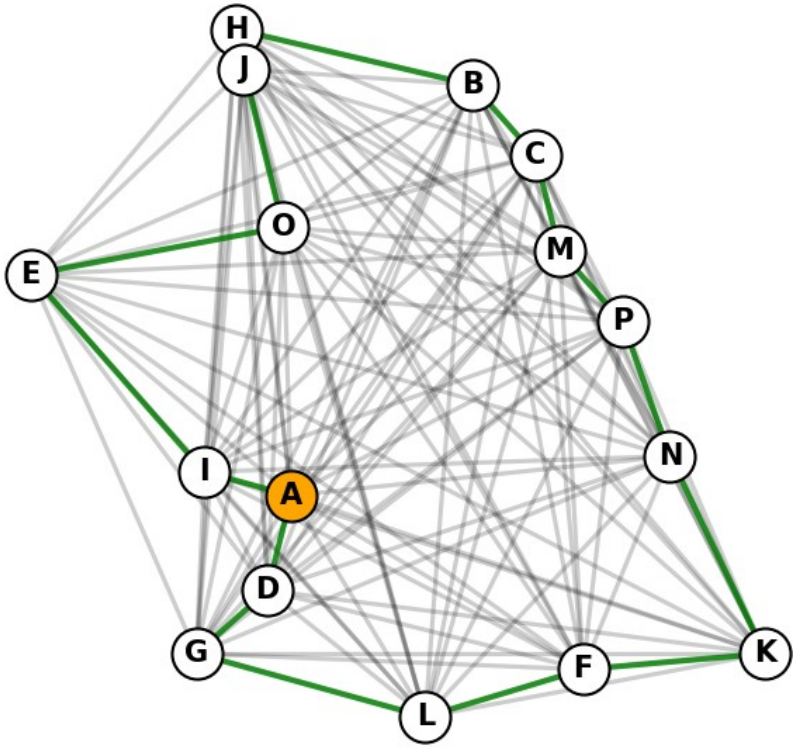


Illustration du choix d'un sommet

Premiers résultats

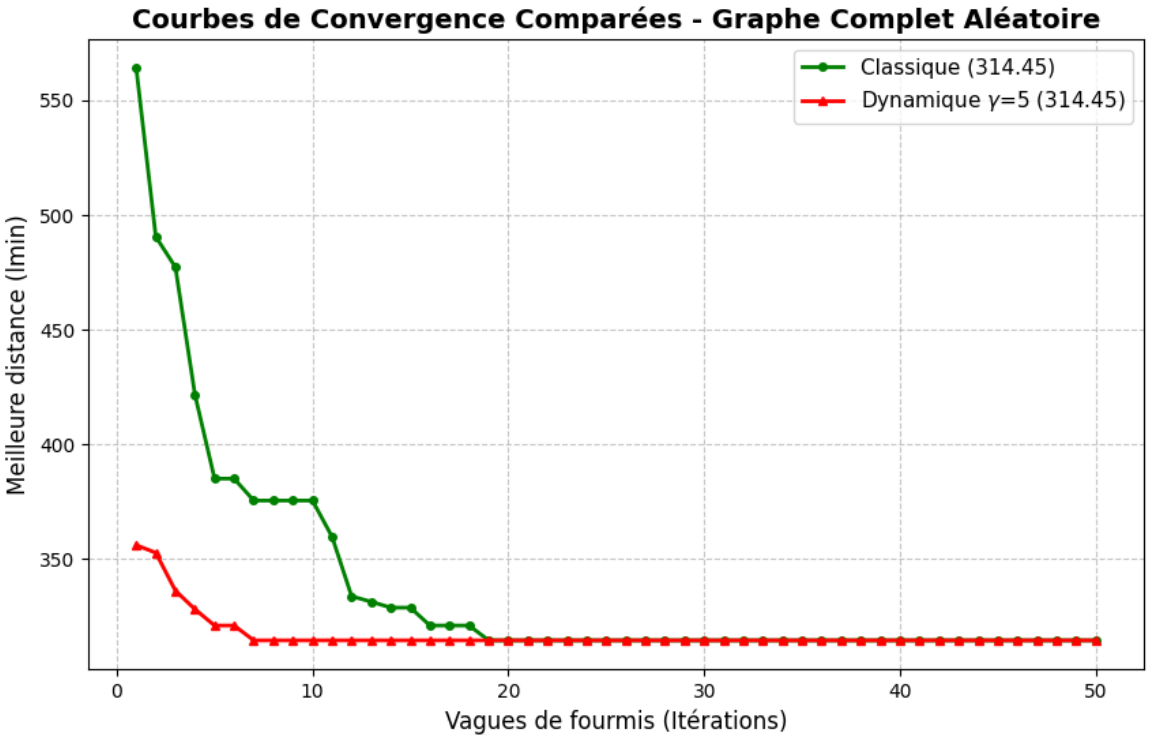


Sommets visités par les fourmis

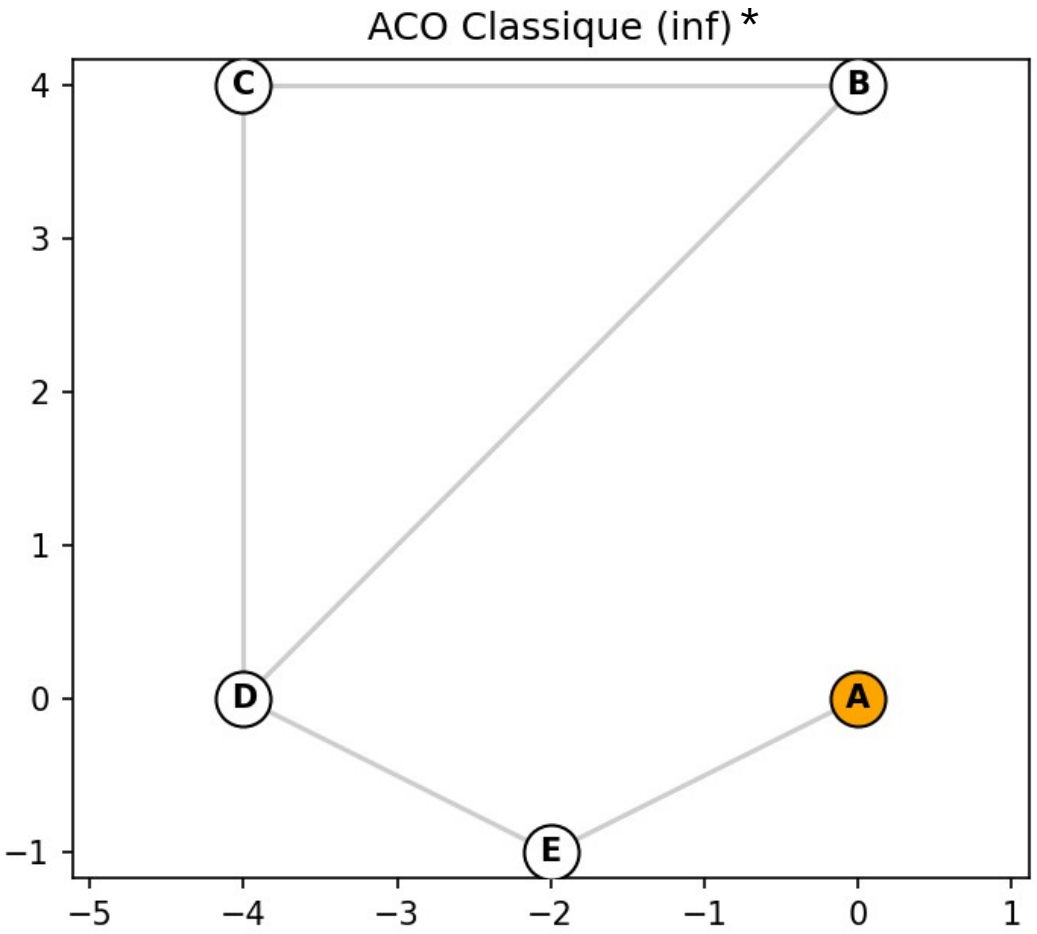
$\alpha = 0.5$

$\beta = 0.5$

Évolution de la distance en fonction du nombre de vagues



Problème : impasses



Sommets visités par les fourmis
**inf= Aucun cycle trouvé*

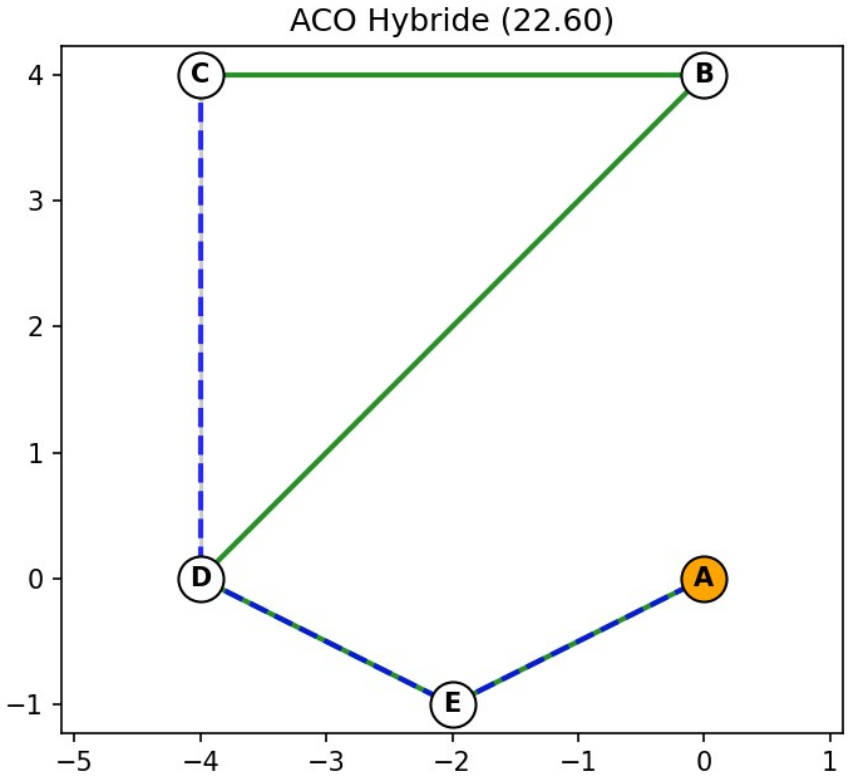
Fourmi X
Départ: {A}
Chemin: A→ E→ D→ B→C

Tous les sommets visités,
retour au sommet de départ

C non voisin de A

Retour impossible,
Chemin supprimé

Approche 3: ACO Hybride



Sommets visités par les fourmis

- Principe: Aller via ACO classique, retour via Dijkstra
- Semi-probabiliste, semi-déterministe

Chemin Aller: A → E → D → B → C
Chemin Retour: C → D → E → A

1) Présentation du problème

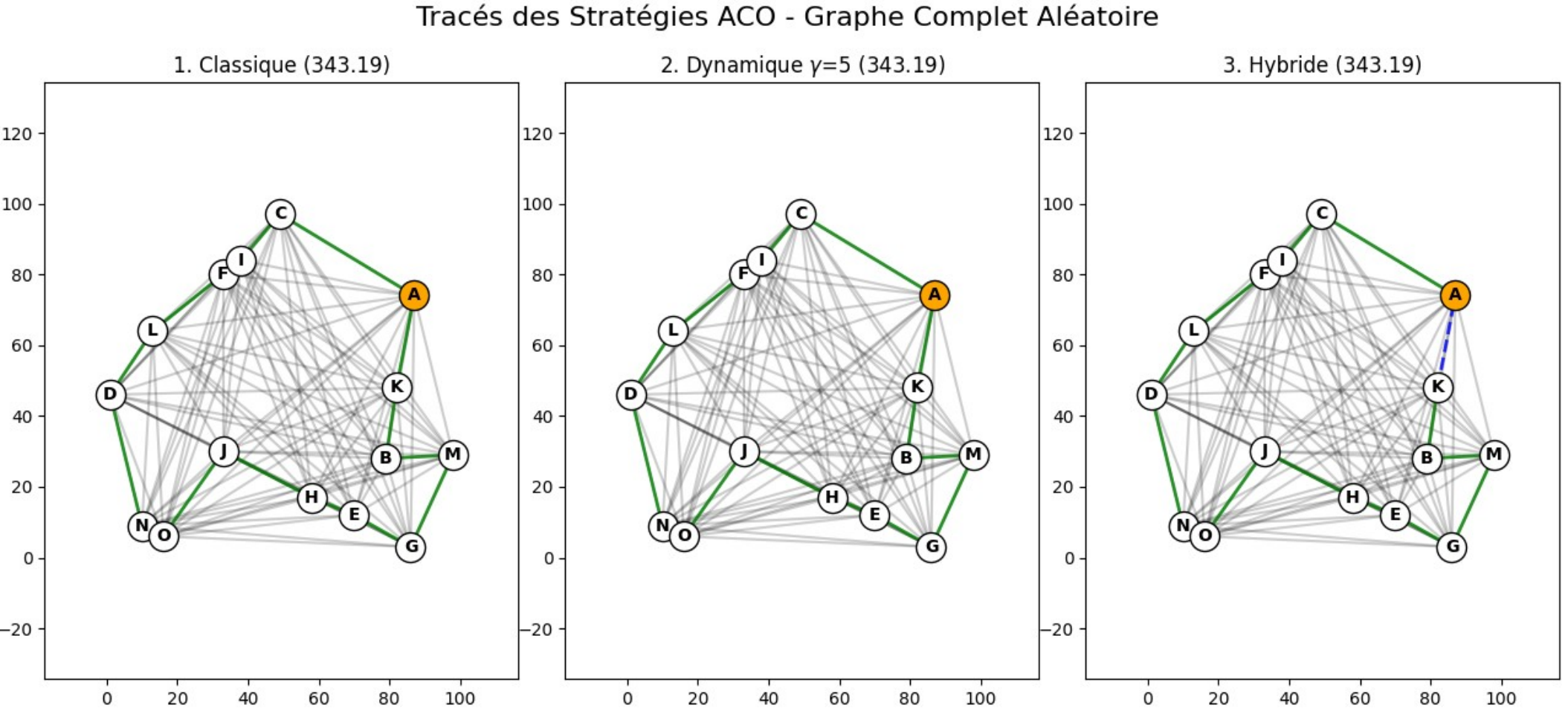
2) 3 approches



3) Résultats

4) Conclusion

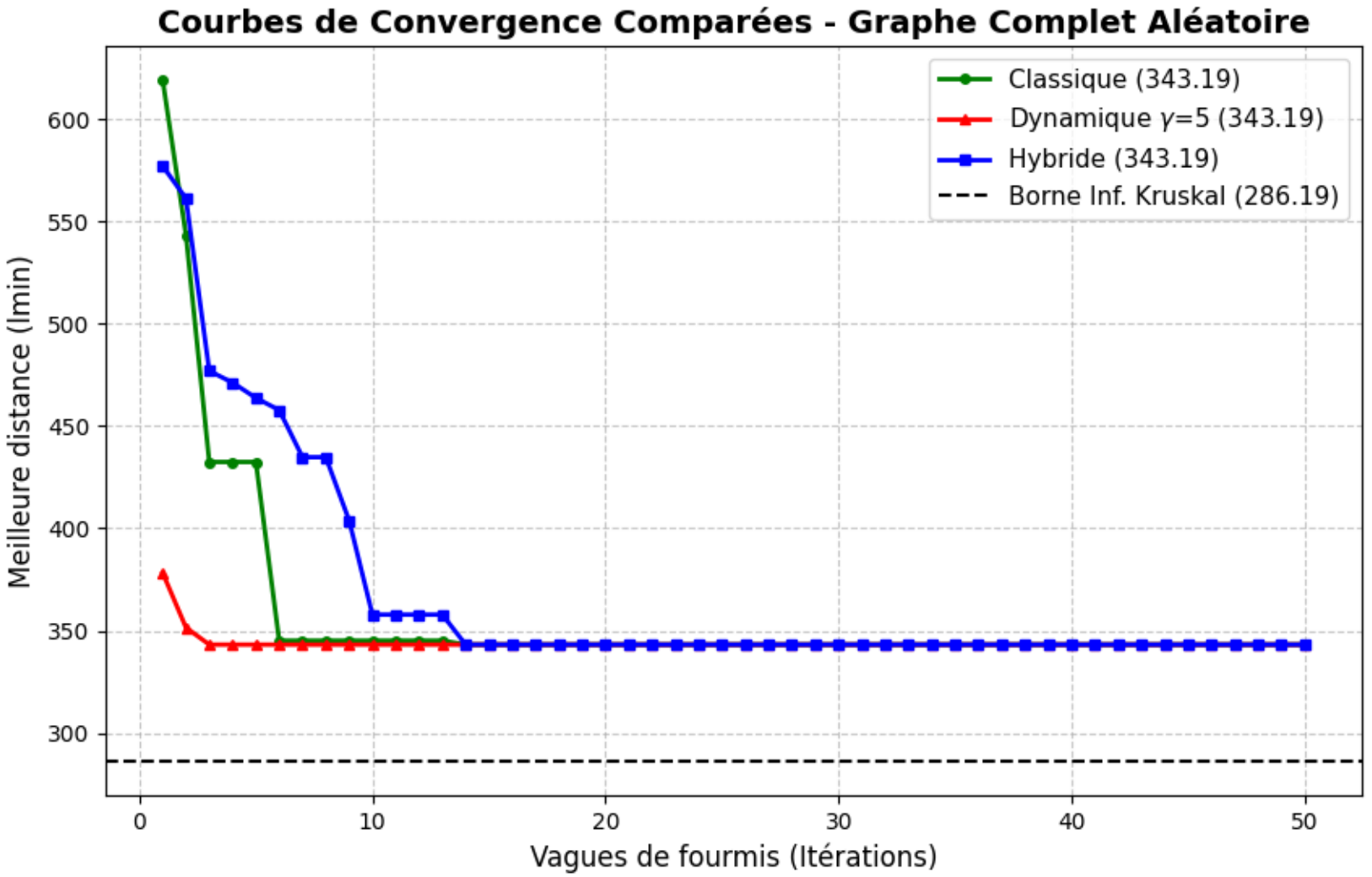
Résultats



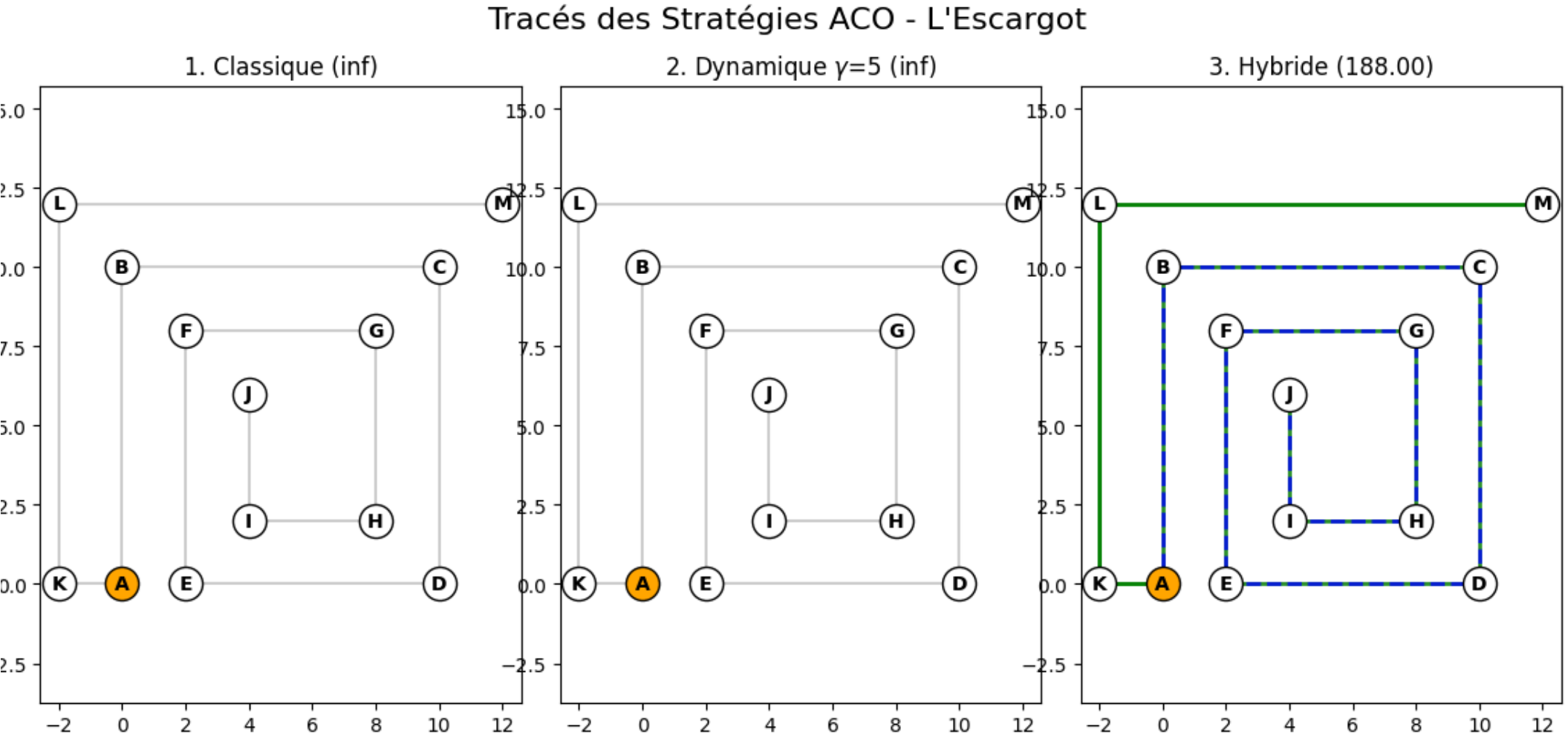
$\alpha = 0.5$

$\beta = 0.5$

Résultats


Courbes de Convergence Comparées - Graphe Complet Aléatoire
Classique (343.19)
Dynamique γ=5 (343.19)
Hybride (343.19)
Borne Inf. Kruskal (286.19)
Meilleure distance (lmin)
Vagues de fourmis (Itérations)
600
550
500
450
400
350
300
0
10
20
30
40
50

Résultats



$\alpha = 0.5$

$\beta = 0.5$

Conclusion

- Rupture majeure de l’ACO Classique.
- ACO Hybride: 100 % de complétion.



Source : Urban-driver.com

Bibliographie

- [1] ROSEAUX : ***Exercices et problèmes résolus de recherche opérationnelle - Tome 1*** : Editions Dunod, ISBN:978-2100493371, 2005
- [2] DORIGO M. AND STÜTZLE T. : ***Ant Colony Optimization*** : MIT Press (2004)
- [3] KRUSKAL J. B. : ***On the shortest spanning subtree of a graph and the traveling salesman problem*** : American Mathematical Society, 1956, Pages 48-50
- [4] ROSENKRANTZ, D. J., STEARNS, R. E., AND LEWIS, P. M. : ***An analysis of several heuristics for the traveling salesman problem*** : SIAM Journal on Computing, 1977

Annexes

1) Complexités
2) Influence des paramètres α et β
3) Quelques résultats d’ACO
4) Script

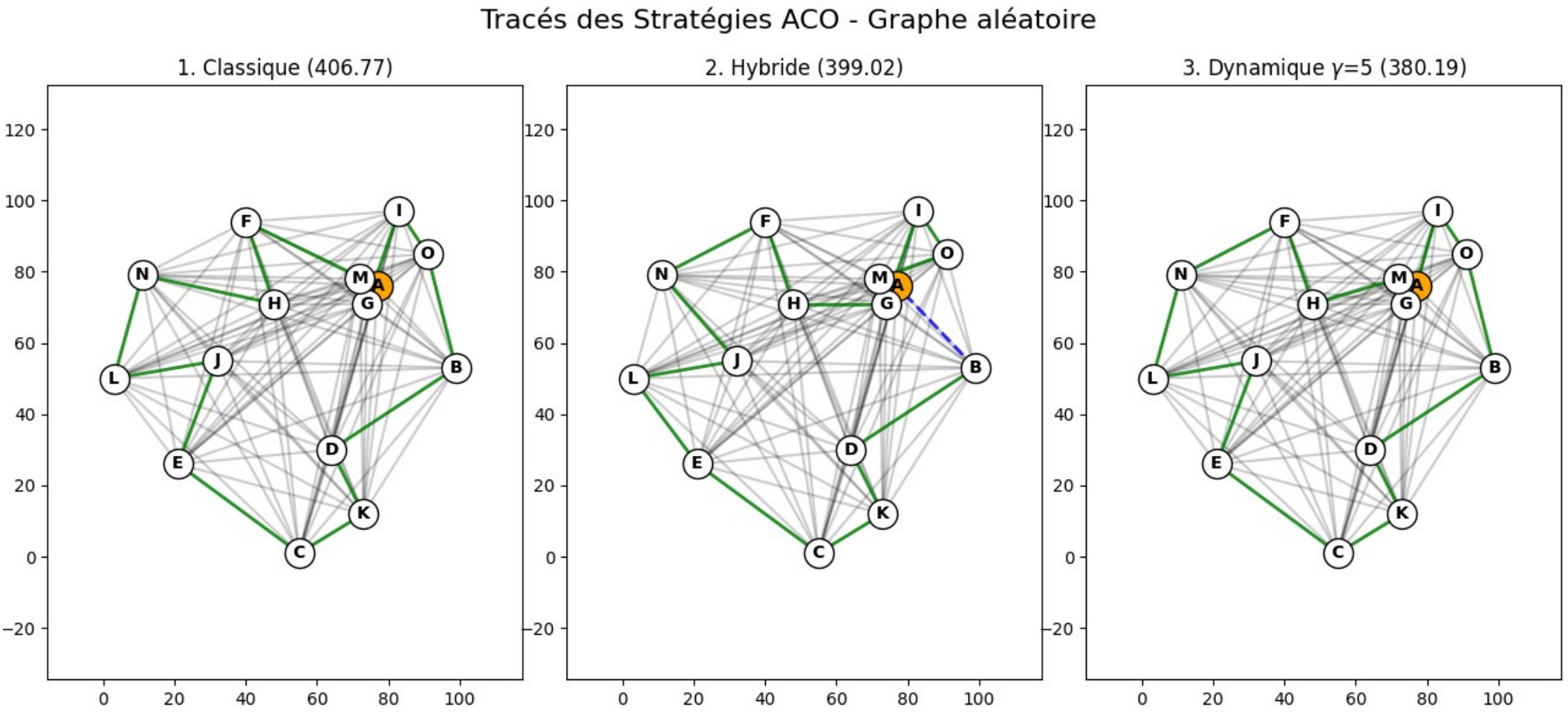
Complexités

N_v : *Nombre de vagues*
N : *Nombre de fourmis par vague*
n : *Nombre total de villes*
k : *Degré moyen d'un sommet*

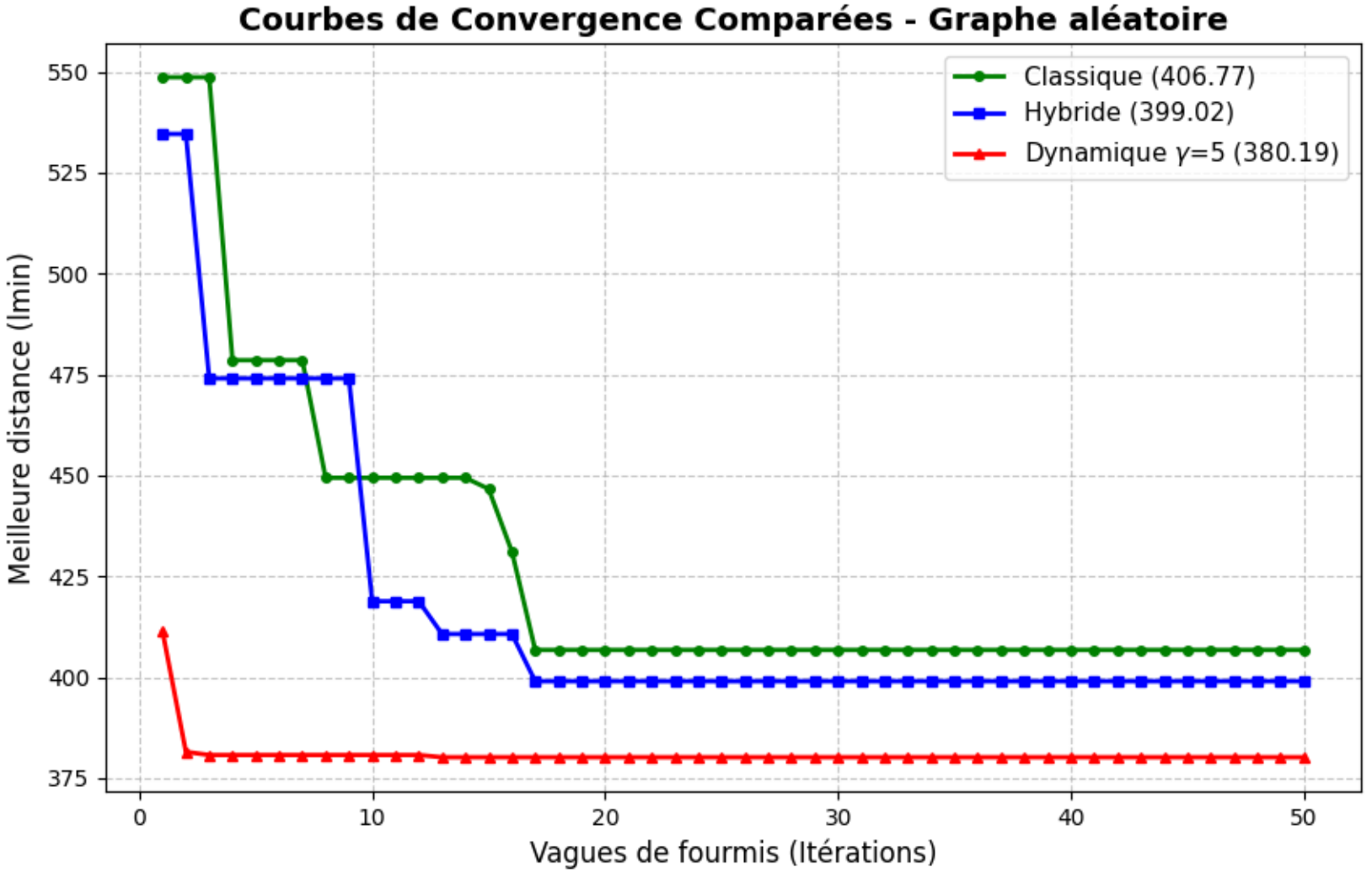
Stratégies algorithmiques	Complexité générale	Graphe complet (k= n-1)
ACO Classique	$O(N_v \cdot N \cdot n^2 \cdot k)$	$O(N_v \cdot N \cdot n^3)$
ACO Dynamique	$O(N_v \cdot N \cdot n^2 \cdot k)$	$O(N_v \cdot N \cdot n^3)$
ACO Hybride	$O(N_v \cdot N \cdot n^2 \cdot (k+1))$	$O(N_v \cdot N \cdot n^3)$

Influence des paramètres α et β

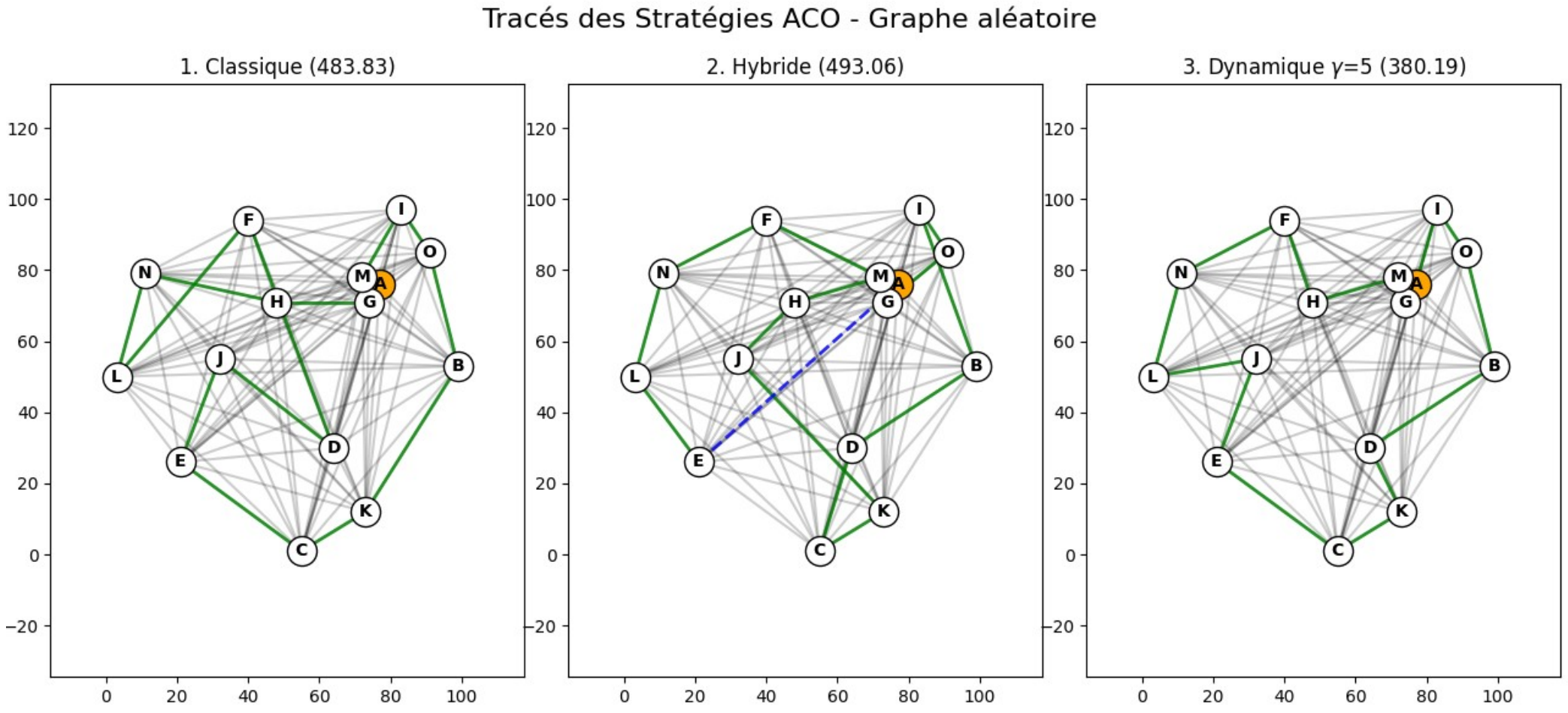
α=β=0.8


Tracés des Stratégies ACO - Graphe aléatoire
1. Classique (406.77)
2. Hybride (399.02)
3. Dynamique γ=5 (380.19)
A
B
C
D
E
F
G
H
I
J
K
L
M
N
O
120
100
80
60
40
20
0
−20
0
20
40
60
80
100

α=β=0.8


Courbes de Convergence Comparées - Graphe aléatoire
Classique (406.77)
Hybride (399.02)
Dynamique γ=5 (380.19)
Meilleure distance (lmin)
Vagues de fourmis (Itérations)
550
525
500
475
450
425
400
375
0
10
20
30
40
50

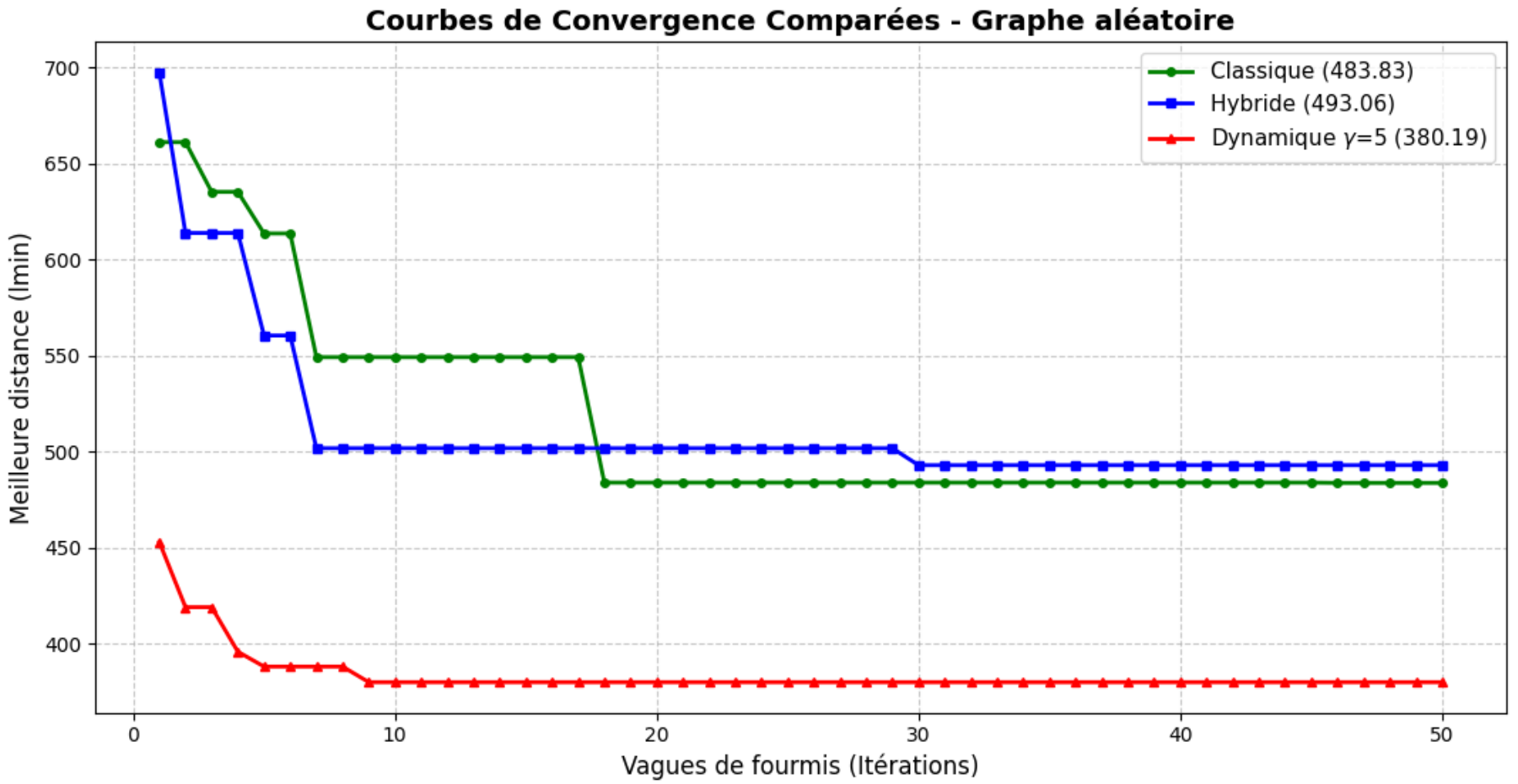
Plus d’importance aux phéromones



$\alpha = 1$

$\beta = 0.1$

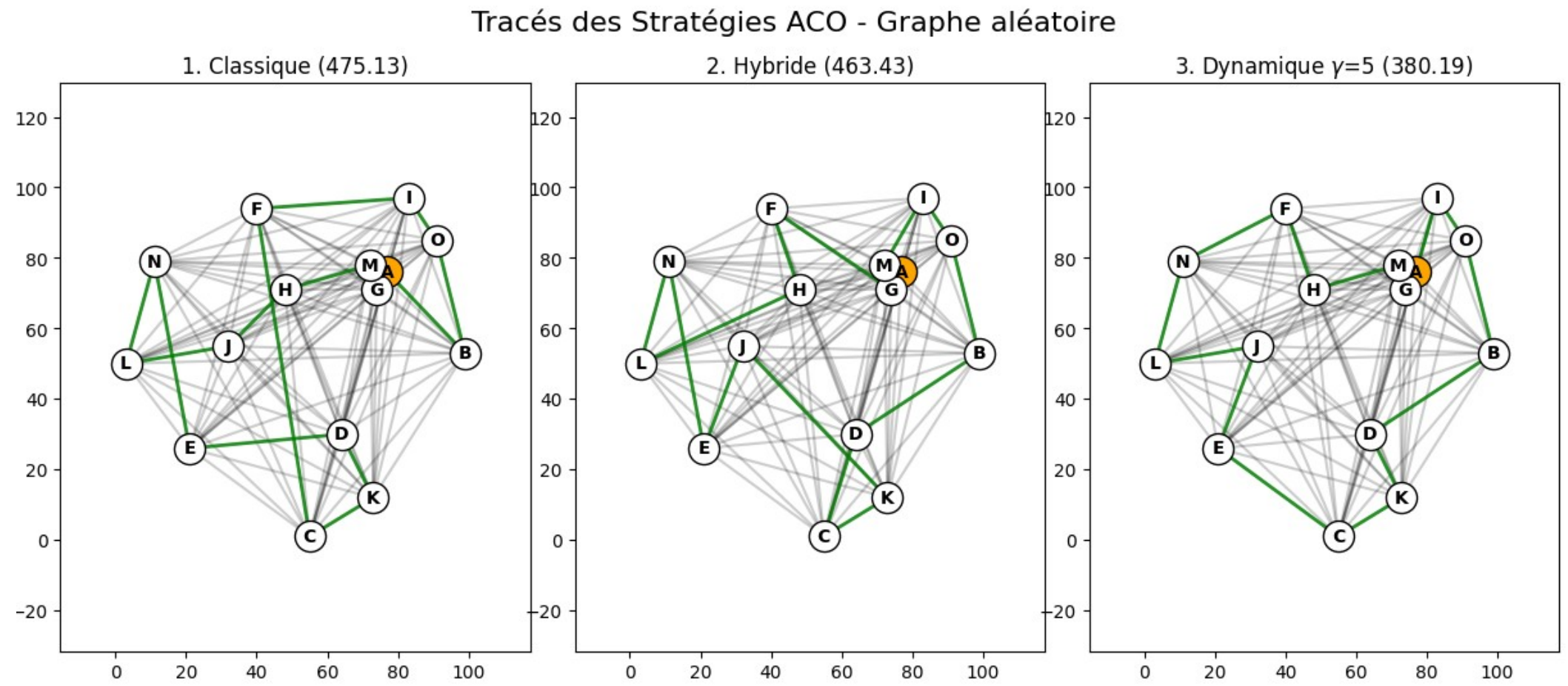
Plus d’importance aux phéromones



$\alpha = 1$

$\beta = 0.1$

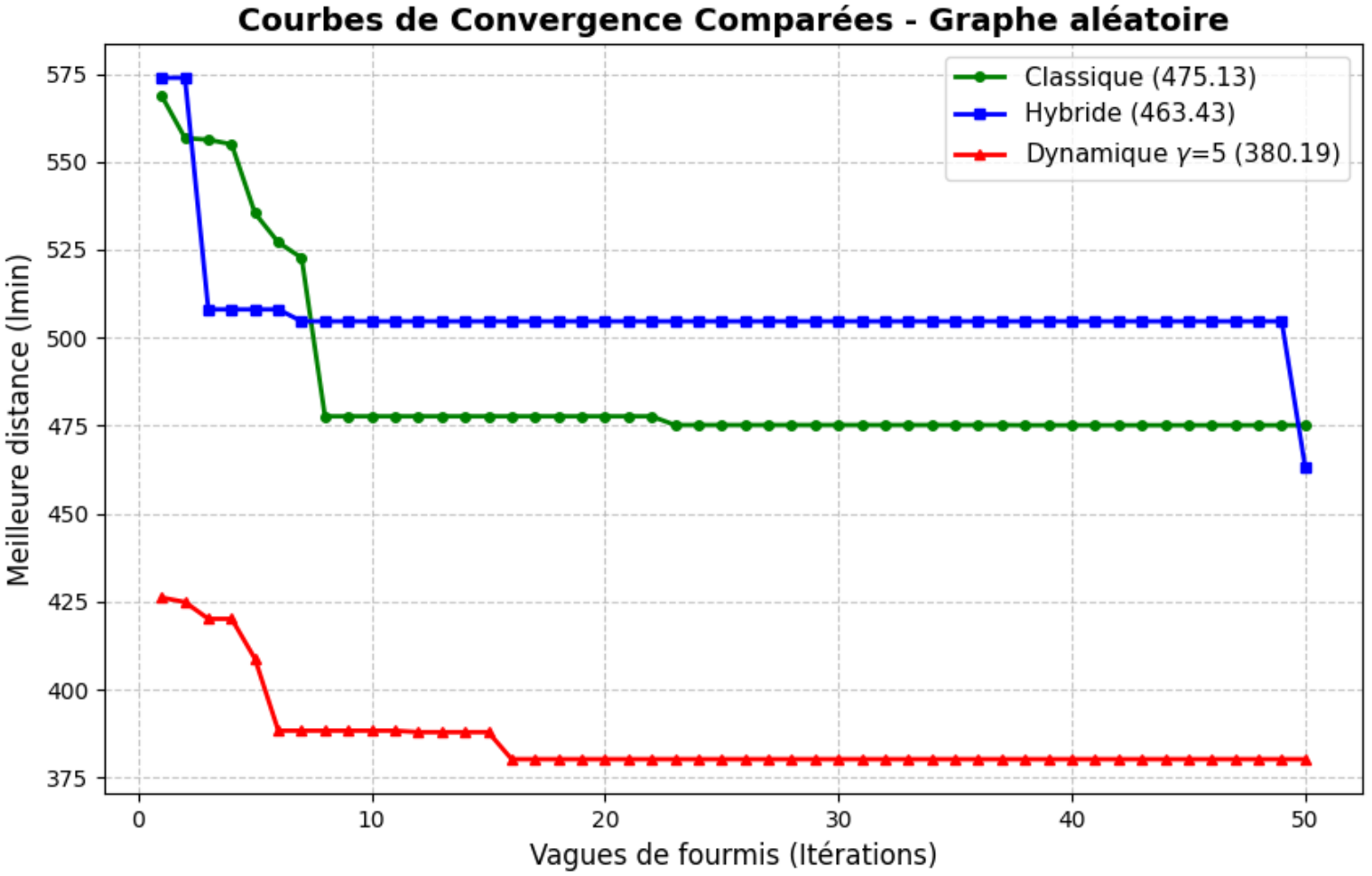
Plus d’importance aux distances courtes



$\alpha = 0.1$

$\beta = 1$

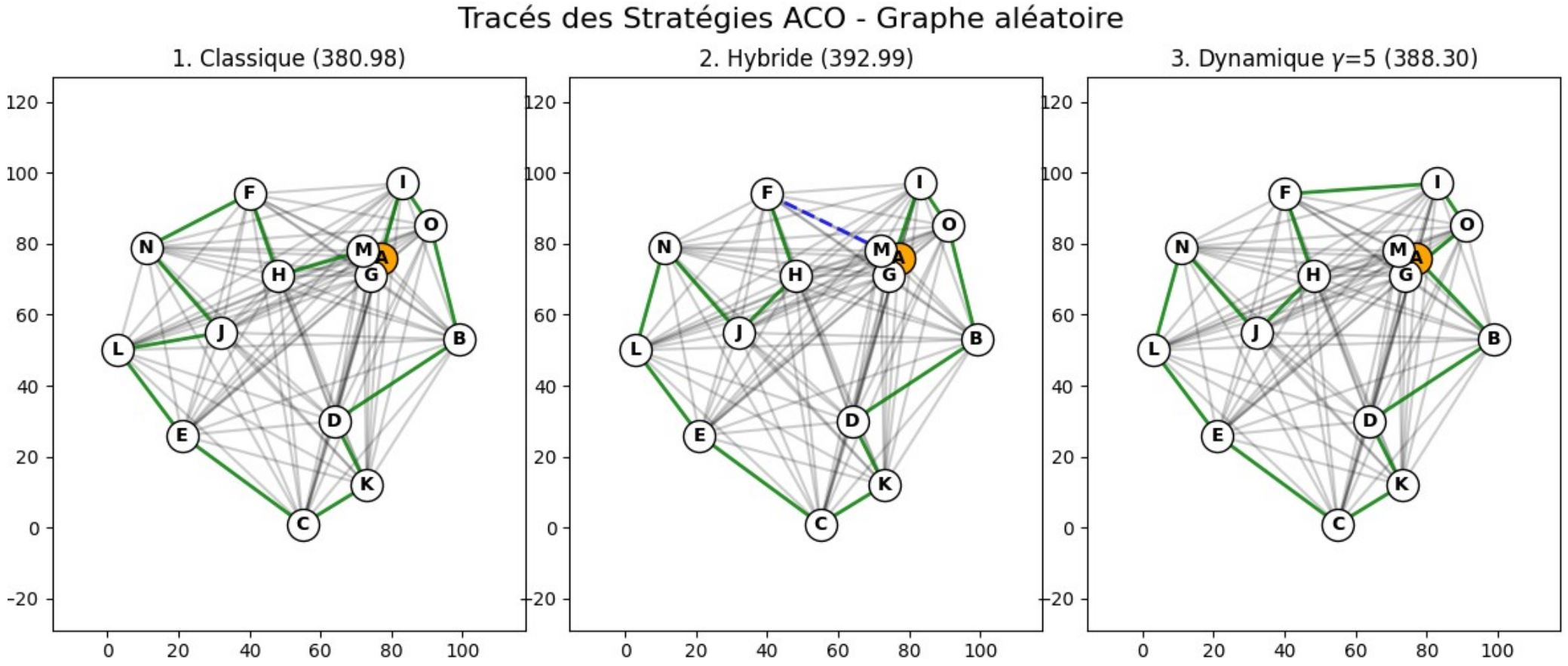
Plus d’importance aux distances courtes



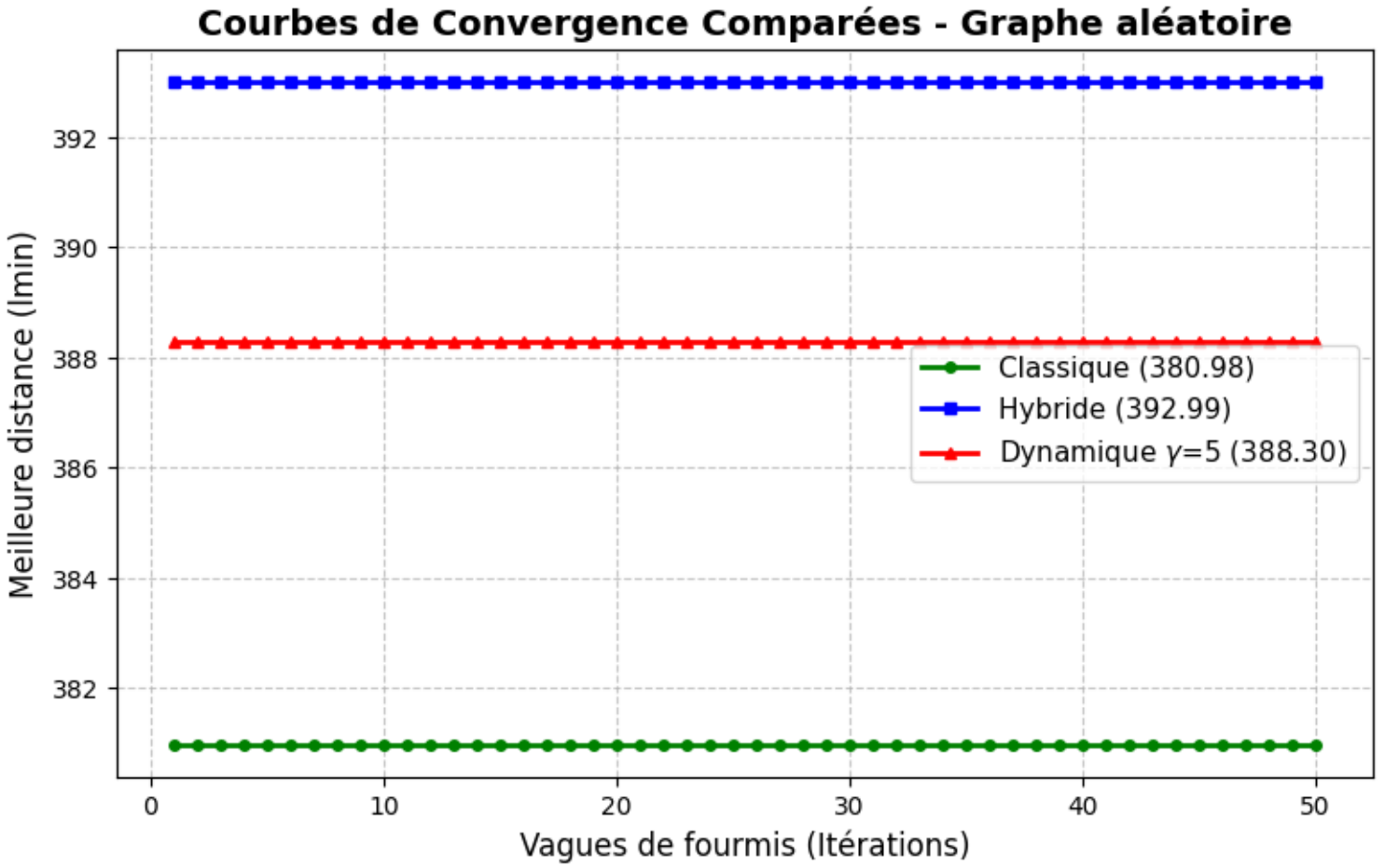
$\alpha = 0.1$

$\beta = 1$

α=β=10

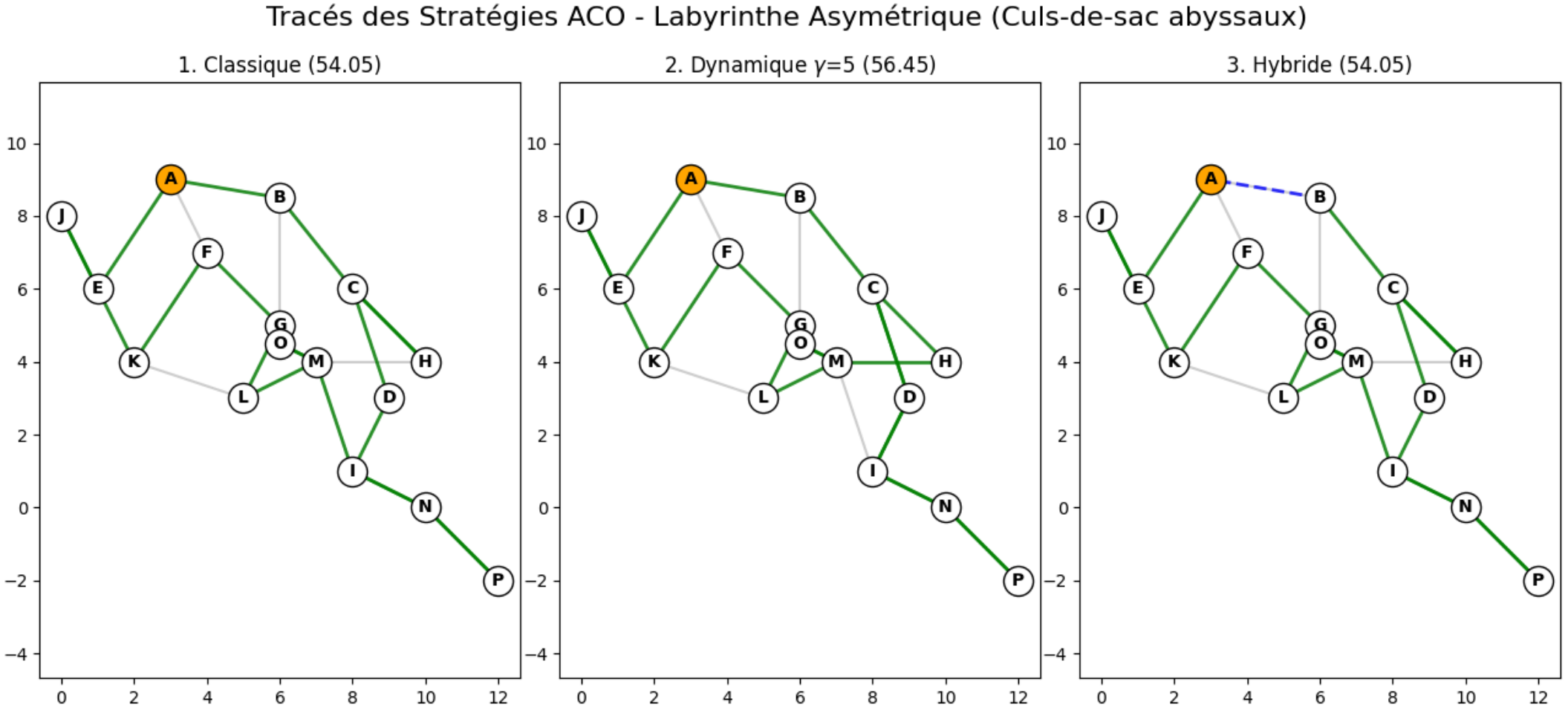

Tracés des Stratégies ACO - Graphe aléatoire
1. Classique (380.98)
2. Hybride (392.99)
3. Dynamique γ=5 (388.30)
A
B
C
D
E
F
G
H
I
J
K
L
M
N
O
120
100
80
60
40
20
0
-20
0
20
40
60
80
100

α=β=10


Courbes de Convergence Comparées - Graphe aléatoire
Meilleure distance (lmin)
392
390
388
386
384
382
0
10
20
30
40
50
Vagues de fourmis (Itérations)
Classique (380.98)
Hybride (392.99)
Dynamique γ=5 (388.30)

Quelques résultats

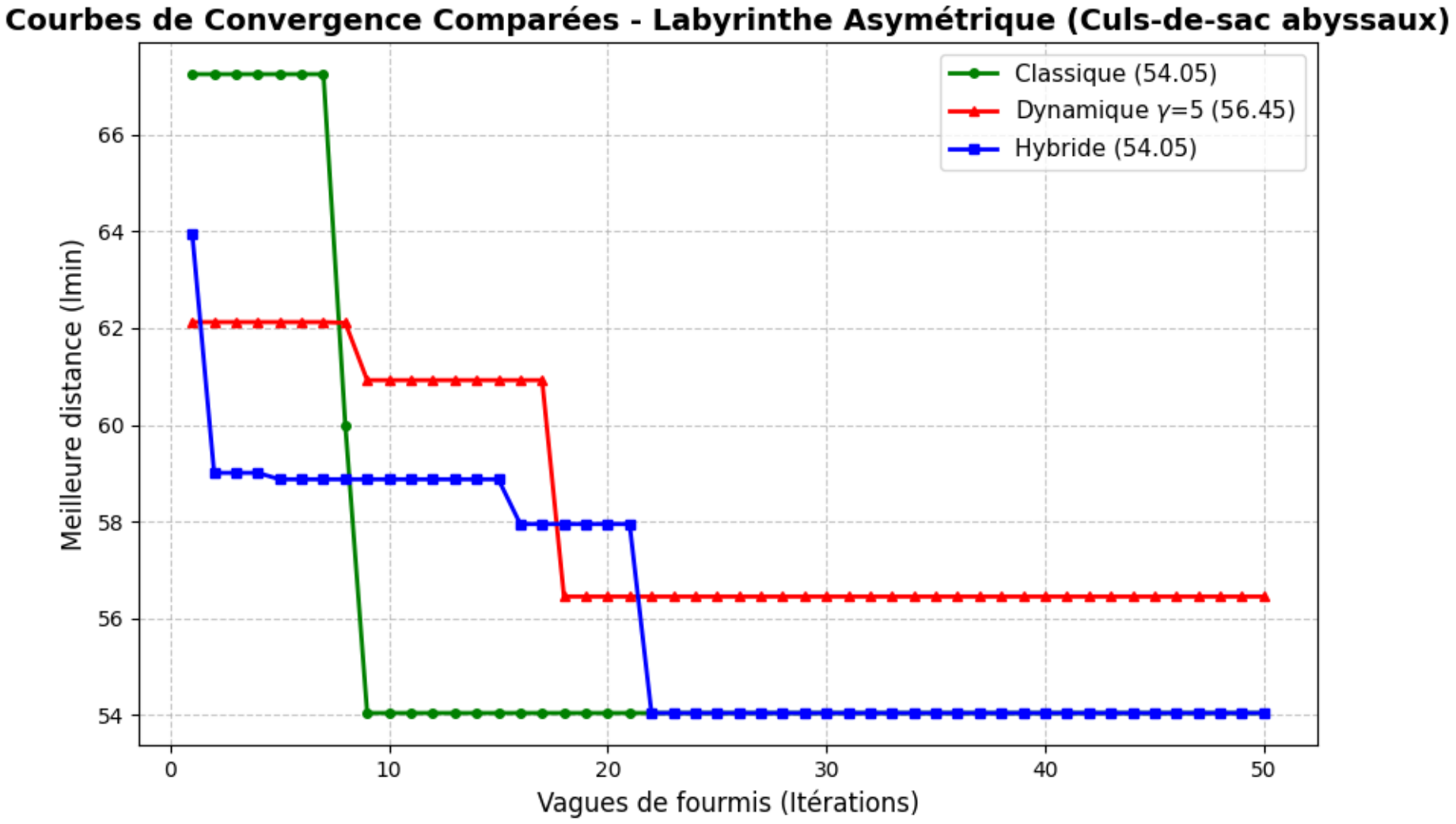
Illustration impasses



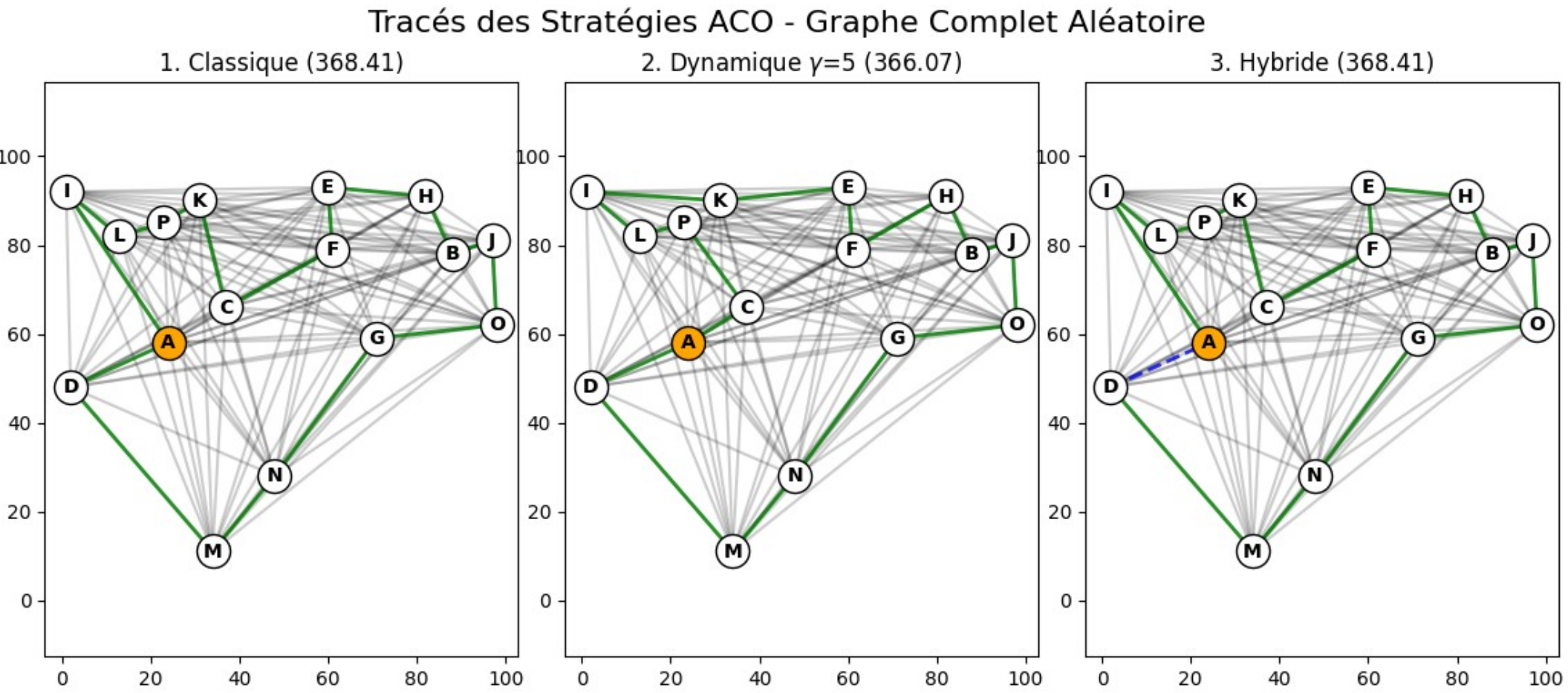
$\alpha = 1$

$\beta = 1$

Illustration impasses



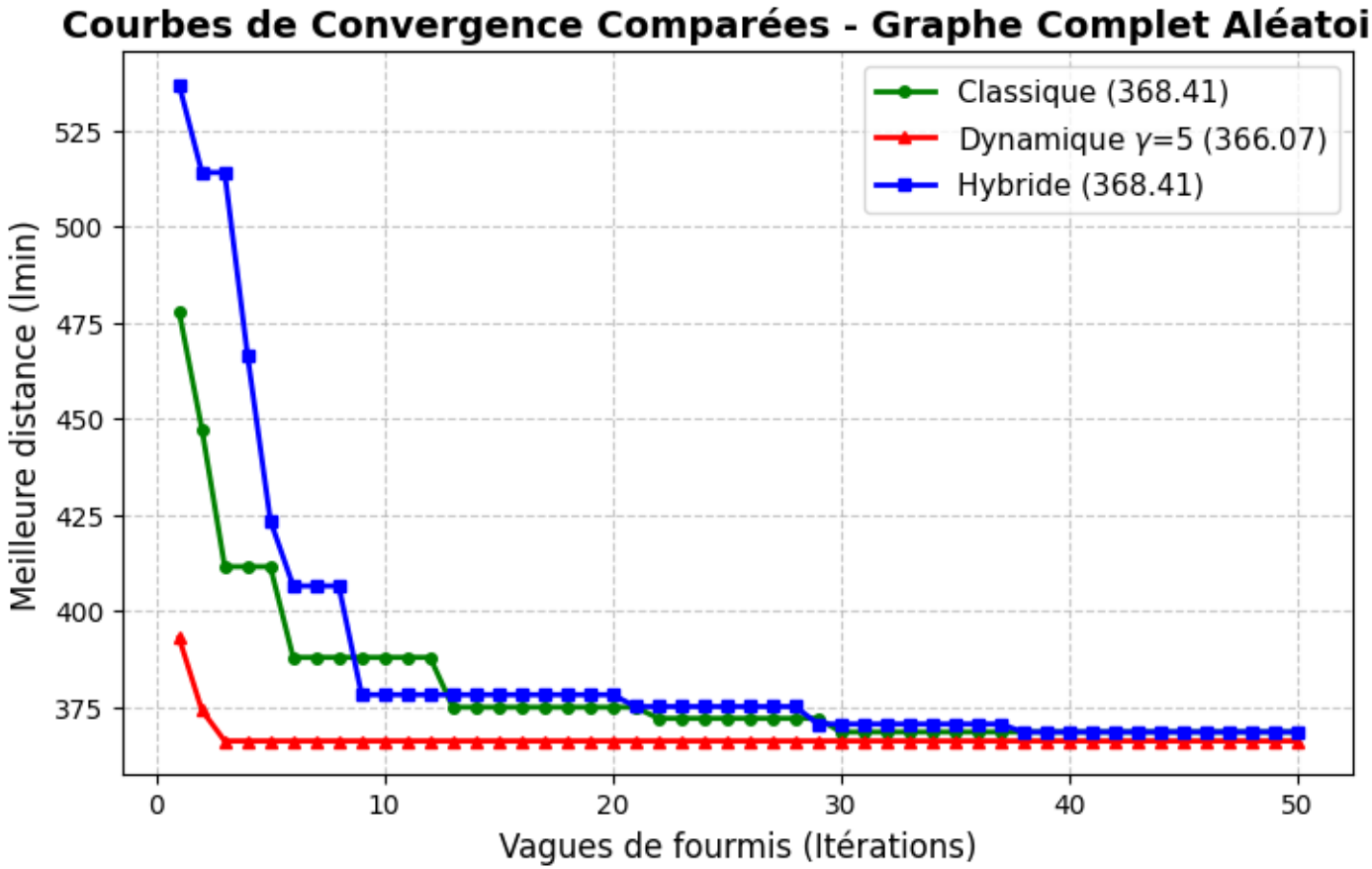
Victoire ACO Dynamique



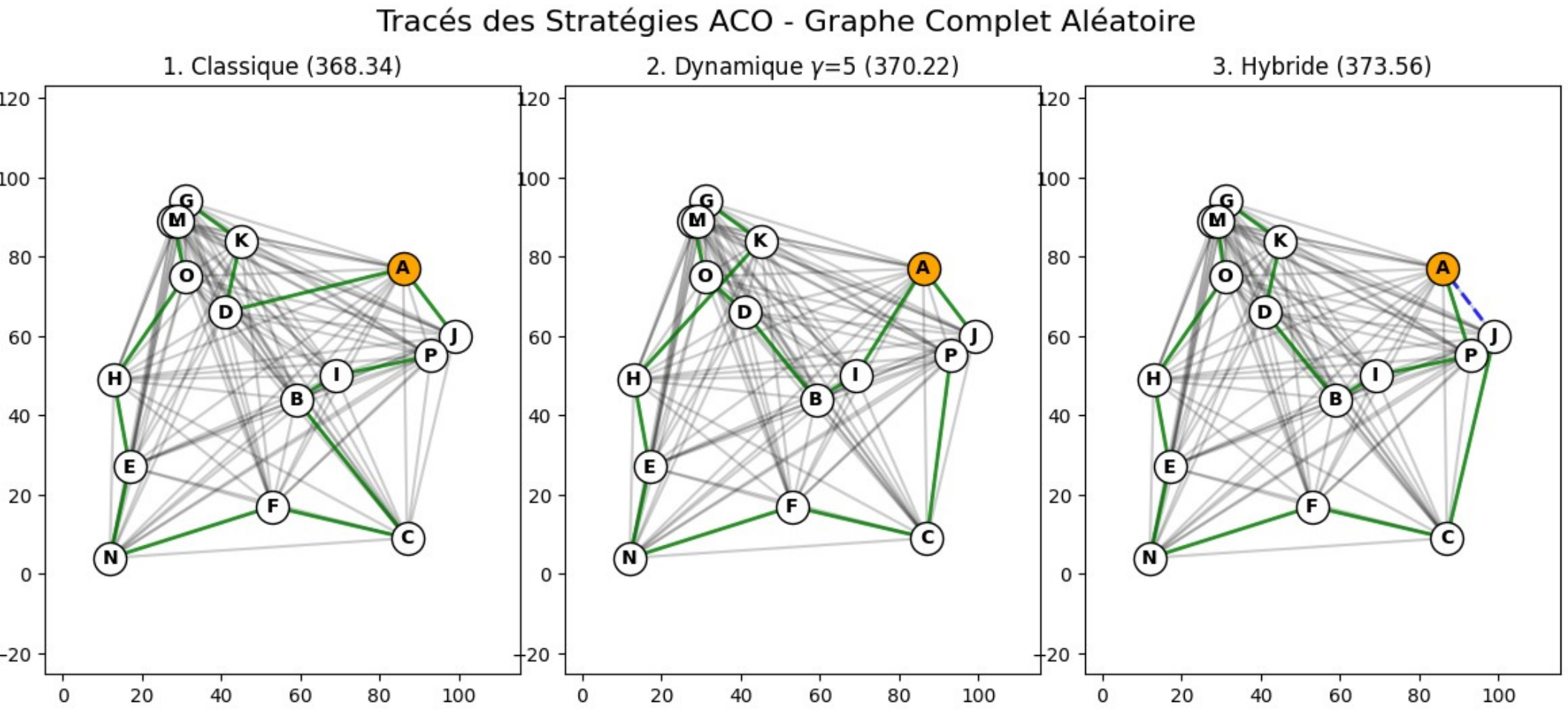
$\alpha = 1$

$\beta = 1$

Victoire ACO Dynamique



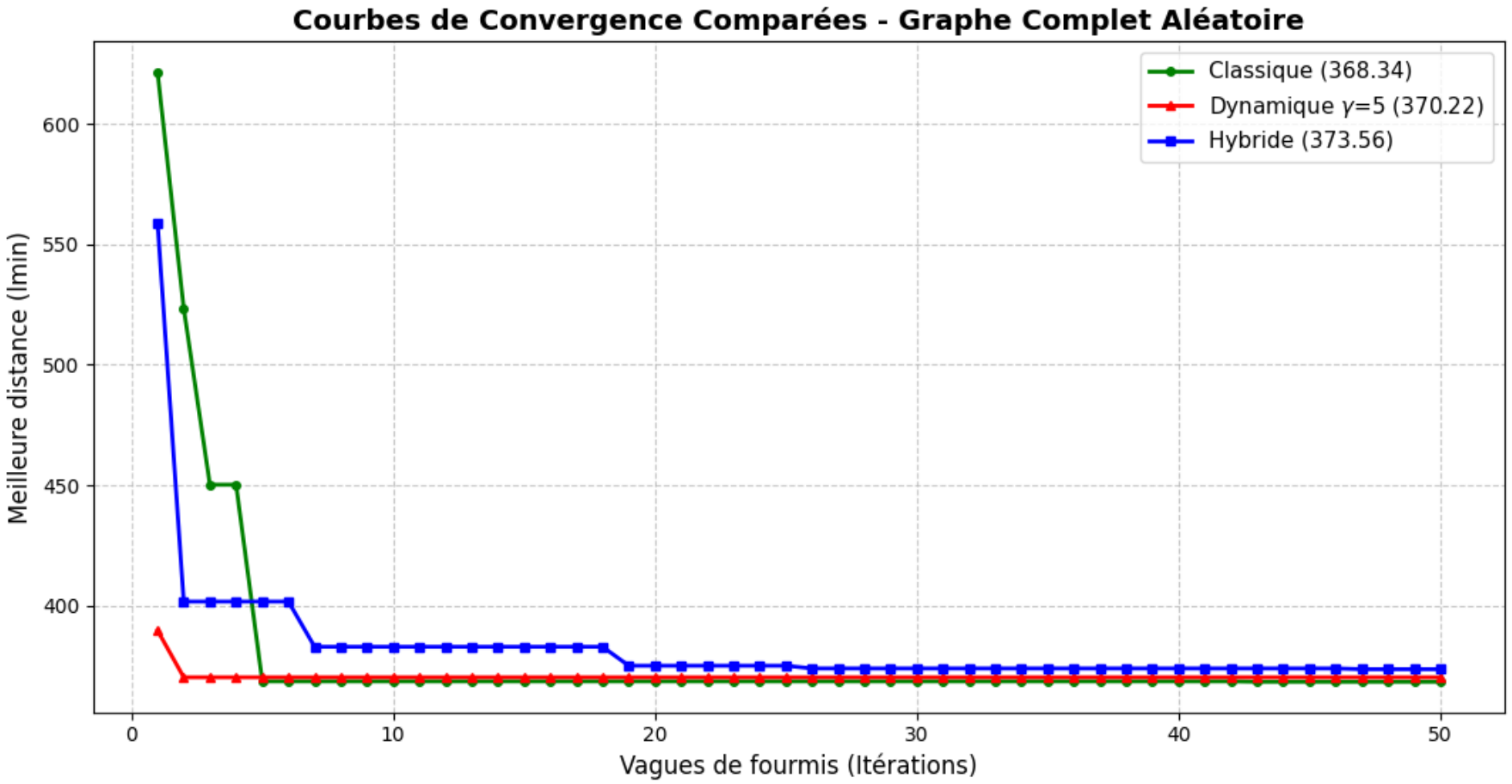
Victoire ACO Classique



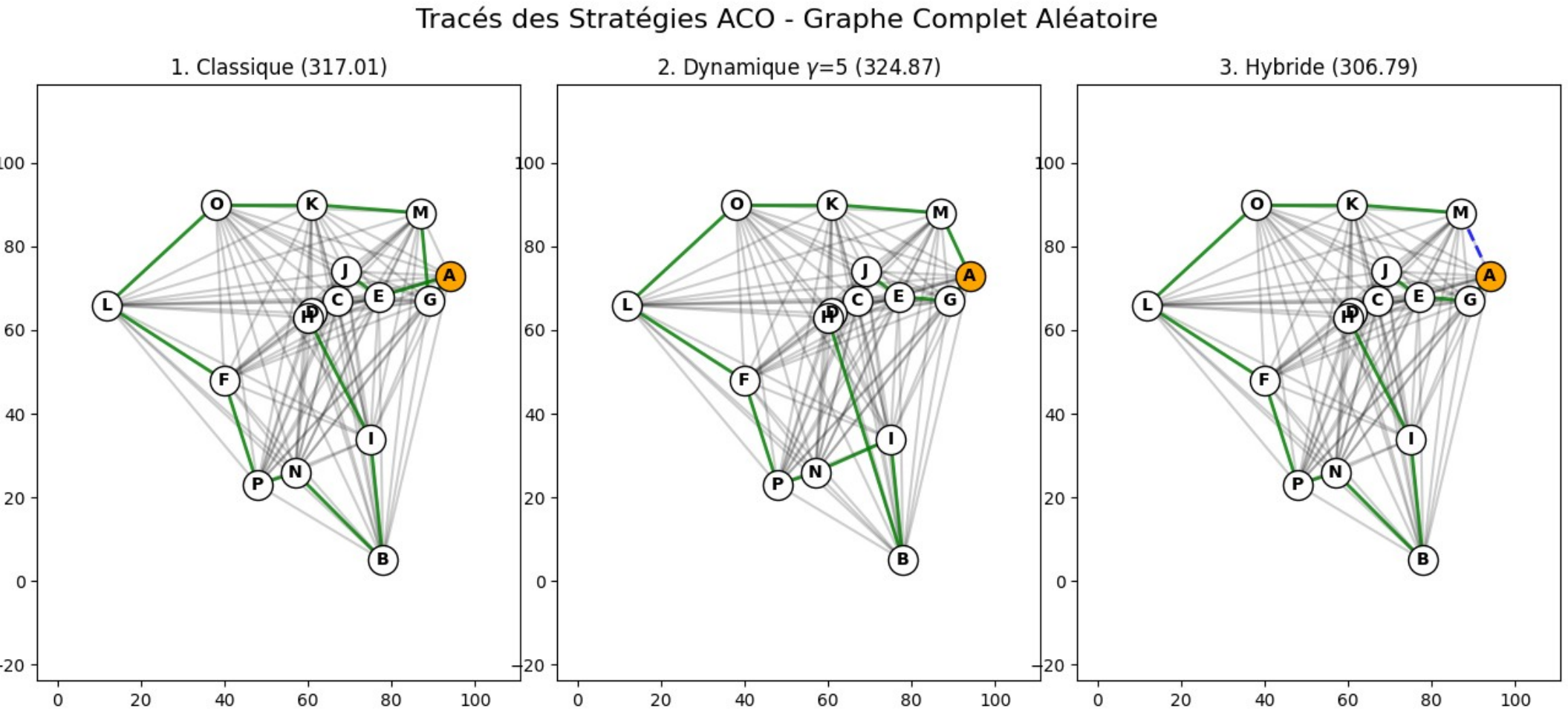
$\alpha = 1$

$\beta = 1$

Victoire ACO Classique



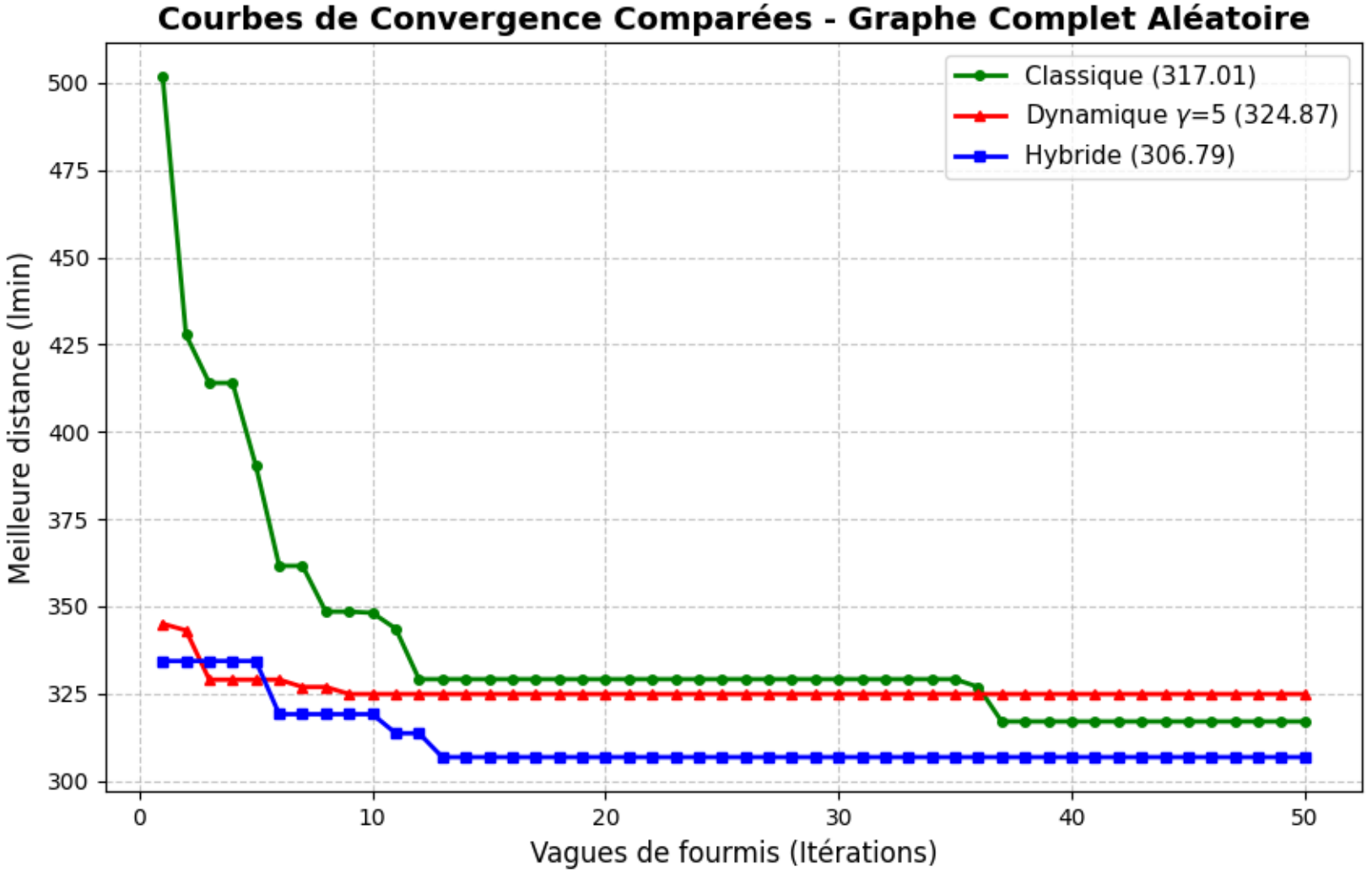
Victoire ACO Hybride



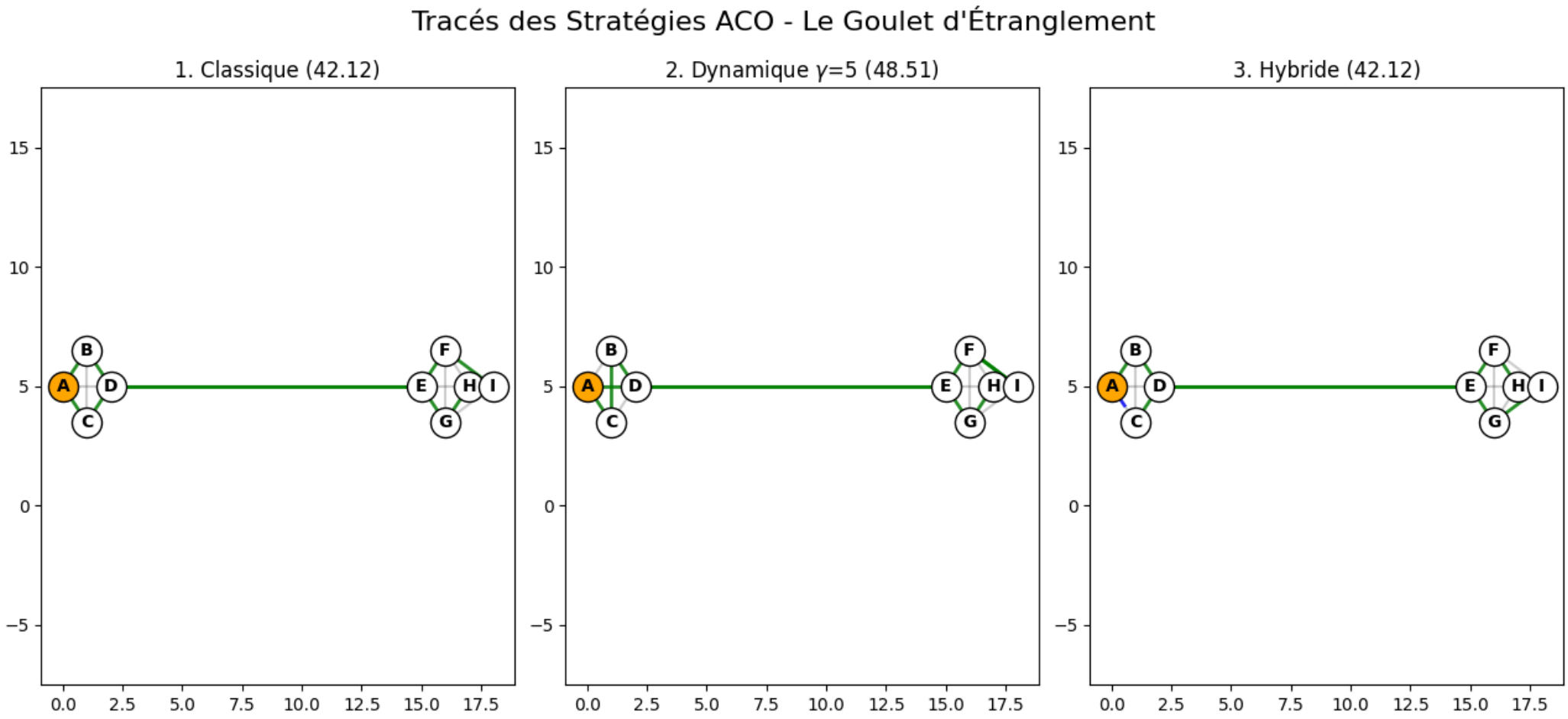
$\alpha = 1$

$\beta = 1$

Victoire ACO Hybride



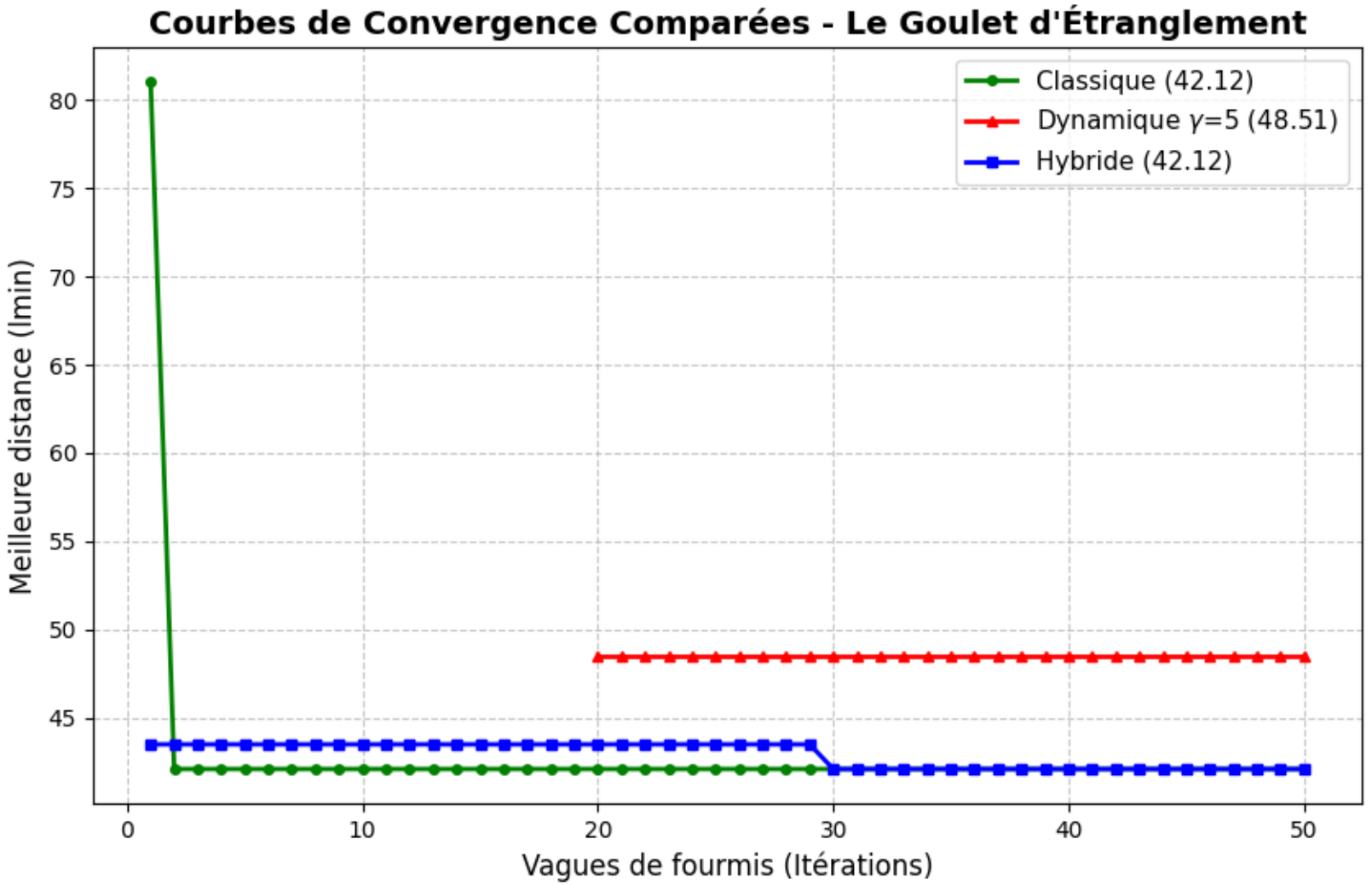
Problèmes ACO Dynamique



$\alpha = 1$

$\beta = 1$

Problèmes ACO Dynamique



Script Python

```python
# TSP Jury.py

# ======================================================
# MODULES
# ======================================================
import matplotlib.pyplot as plt
import numpy as np
import random

# ======================================================
# 1. CATALOGUE DES GRAPHES
# ======================================================

# --- Graphe complet aléatoire (16 villes sur grille 100x100) ---
NB_VILLES_ALEA = 16                    # MAX avant crash: 32

clients_complet = [chr(i) for i in range(65, 65 + NB_VILLES_ALEA)]   # ['A', 'B', ...]
coords_complet  = {v: (random.randint(0, 100), random.randint(0, 100)) for v in clients_complet}
graphe_complet  = {v: [u for u in clients_complet if u != v] for v in clients_complet}

CATALOGUE_GRAPHES = {
    1: {"nom": "Circuit linéaire",
        "Graphe": {'A': ['E'], 'B': ['C', 'D'], 'C': ['B', 'D'], 'D': ['E','C', 'B'], 'E': ['D', 'A']},
        "coords": {'A': (0, 0), 'B': (0, 4), 'C': (-4, 4), 'D': (-4, 0), 'E': (-2, -1)}},
    2: {"nom": "L'anneau'",
        "Graphe": {'A': ['B', 'F'], 'B': ['A', 'C', 'E'], 'C': ['B', 'D'], 'D': ['C', 'E'], 'E': ['D', 'B', 'F'], 'F': ['A', 'E']},
        "coords": {'A': (0, 0), 'B': (2, 3), 'C': (5, 3), 'D': (6, 0), 'E': (3, -2), 'F': (1, -2)}},
    3: {"nom": "Le pont",
        "Graphe": {'A': ['B', 'C'], 'B': ['A', 'C', 'D'], 'C': ['A', 'B'], 'D': ['B', 'E'], 'E': ['D', 'F', 'G'], 'F': ['E', 'G'], 'G': ['E', 'F', 'H'], 'H': ['G']},
        "coords": {'A': (0, 4), 'B': (2, 5), 'C': (4, 4), 'D': (6, 4), 'E': (8, 3), 'F': (9, 5), 'G': (10, 3), 'H': (12, 3)}},
    4: {"nom": "Piège local",
        "Graphe": {'A': ['B', 'J'], 'B': ['A', 'C'], 'C': ['B', 'D', 'I'], 'D': ['C', 'E'], 'E': ['D', 'F'], 'F': ['E', 'G', 'H'], 'G': ['F'], 'H': ['F', 'I'], 'I': ['H', 'C', 'J'], 'J': ['I', 'A']},
        "coords": {'A': (0, 2), 'B': (1, 4), 'C': (3, 5),
```



```
'D': (5, 4), 'E': (6, 2), 'F': (5, 0), 'G': (3, -1), 'H': (1, 0), 'I': (2, 2), 'J': (1, 1)}},
    5: {"nom": "Labyrinthe symétrique",
        "Graphe": {'A': ['B', 'D'], 'B': ['A', 'C', 'E'], 'C': ['B', 'F'], 'D': ['A', 'E', 'G'], 'E': ['B', 'D', 'F', 'H'], 'F': ['C', 'E', 'I'], 'G': ['D', 'H'], 'H': ['G', 'E', 'I', 'J'], 'I': ['F', 'H', 'K'], 'J': ['H', 'L'], 'K': ['I', 'L'], 'L': ['J', 'K']},
        "coords": {'A': (0, 6), 'B': (2, 7), 'C': (4, 6), 'D': (0, 4), 'E': (2, 5), 'F': (4, 4), 'G': (0, 2), 'H': (2, 3), 'I': (4, 2), 'J': (2, 1), 'K': (4, 0), 'L': (6, 1)}},
    6: {"nom": "Le Losange",
        "Graphe": {'A': ['B', 'E', 'D'], 'B': ['C', 'E', 'A'], 'C': ['B', 'D', 'E'], 'D': ['A', 'E', 'C'], 'E': ['A', 'B', 'C', 'D']},
        "coords": {'A': (0, 0), 'B': (3, 4), 'C': (8, 0), 'D': (3, -4), 'E': (4, 0)}},
    7: {"nom": "Le Goulet d'Étranglement",
        "Graphe": {'A': ['B', 'C', 'D'], 'B': ['A', 'C', 'D'], 'C': ['A', 'B', 'D'], 'D': ['A', 'B', 'C', 'E'], 'E': ['D', 'F', 'G', 'H'], 'F': ['E', 'G', 'H', 'I'], 'G': ['E', 'F', 'H', 'I'], 'H': ['E', 'F', 'G', 'I'], 'I': ['F', 'G', 'H']},
        "coords": {'A': (0, 5), 'B': (1, 6.5), 'C': (1, 3.5), 'D': (2, 5), 'E': (15, 5), 'F': (16, 6.5), 'G': (16, 3.5), 'H': (17, 5), 'I': (18, 5)}},
    8: {"nom": "L'Escargot",
        "Graphe": {'A': ['B', 'K'], 'B': ['A', 'C'], 'C': ['B', 'D'], 'D': ['C', 'E'], 'E': ['D', 'F'], 'F': ['E', 'G'], 'G': ['F', 'H'], 'H': ['G', 'I'], 'I': ['H', 'J'], 'J': ['I'], 'K': ['A', 'L'], 'L': ['K', 'M'], 'M': ['L']},
        "coords": {'A': (0, 0), 'B': (0, 10), 'C': (10, 10), 'D': (10, 0), 'E': (2, 0), 'F': (2, 8), 'G': (8, 8), 'H': (8, 2), 'I': (4, 2), 'J': (4, 6), 'K': (-2, 0), 'L': (-2, 12), 'M': (12, 12)}},
    9: {"nom": "Labyrinthe Asymétrique (Culs-de-sac abyssaux)",
        "Graphe": {'A': ['B', 'E', 'F'], 'B': ['A', 'C', 'G'], 'C': ['B', 'D', 'H'], 'D': ['C', 'I'], 'E': ['A', 'J', 'K'], 'F': ['A', 'G', 'K'], 'G': ['B', 'F', 'L'], 'H': ['C', 'M'], 'I': ['D', 'M', 'N'], 'J': ['E'], 'K': ['E', 'F', 'L'], 'L': ['G', 'K', 'M'], 'M': ['H', 'I', 'L', 'O'], 'N': ['I', 'P'], 'O': ['M'], 'P': ['N']},
        "coords": {'A': (3, 9), 'B': (6, 8.5), 'C': (8, 6), 'D': (9, 3), 'E': (1, 6), 'F': (4, 7), 'G': (6, 5), 'H': (10, 4), 'I': (8, 1), 'J': (0, 8), 'K': (2, 4), 'L': (5, 3), 'M': (7, 4), 'N': (10, 0), 'O': (6, 4.5), 'P': (12, -2)}},
    10: {'nom': "Graphe Complet Aléatoire",
         'Graphe': graphe_complet,
         'coords': coords_complet,
```

```
050|          'LISTE_CLIENTS': clients_complet},
051|     11: {'nom': "L'Hydre",
052|          'Graphe': {'A': ['B', 'C', 'D'],'B': ['A', 'E',
'F'],  'C': ['A', 'G', 'H'],  'D': ['A', 'I', 'J'],'E': ['B',
'K'],'F': ['B'],'G': ['C', 'L'],       'H': ['C'],'I':
['D'],'J': ['D', 'M'],'K': ['E'],'L': ['G'],'M': ['J']},
053|          'coords': {'A': (0, 0), 'B': (0, 4),'C': (-4, -2),
'D': (4, -2),   'E': (-2, 6),'F': (2, 6),'G': (-6, -1),'H': (-4,
-5), 'I': (4, -5),'J': (6, -1),'K': (-3, 8),'L': (-8, 1),'M':
(8, -3)},
054|          'LISTE_CLIENTS': ['A', 'B', 'C', 'D', 'E', 'F',
'G', 'H', 'I', 'J', 'K', 'L', 'M']}
055| }
056|
057| # --- Sélection du graphe actif ---
058| NUMERO_GRAPHE = 7
059|
060| Graphe        = CATALOGUE_GRAPHES[NUMERO_GRAPHE]["Graphe"]
061| coords        = CATALOGUE_GRAPHES[NUMERO_GRAPHE]["coords"]
062| LISTE_CLIENTS = list(Graphe.keys())
063| noms          = LISTE_CLIENTS
064|
065|
066| # ====================================================
067| # 2. PRÉPARATION DU GRAPHE
068| # ====================================================
069|
070| def distance_euclidienne(p1, p2):
071|     """Distance L2 entre deux points 2D."""
072|     return np.hypot(p1[0] - p2[0], p1[1] - p2[1])
073|
074|
075| def creer_graphe_pondere(Graphe, coords):
076|     """Graphe simple → graphe pondéré : {u: [(v, dist),
...]}."""
077|     G_pondere = {}
078|     for u in Graphe:
079|         G_pondere[u] = [(v, distance_euclidienne(coords[u],
coords[v])) for v in Graphe[u]]
080|     return G_pondere
081|
082|
083| def cle_min(d, L):
084|     """Indice du minimum de d parmi les clés de L — O(|
L|)."""
085|     cmin, vmin = None, float("inf")
086|     for c in L:
087|         if d[c] < vmin:
088|             vmin, cmin = d[c], c
```



```
089|     return cmin
090|
091|
092| def dijkstra_vers_tous(G_pondere, depart):
093|     """
094|     Plus courts chemins depuis 'depart' vers tous les
sommets.
095|     Implémentation par extraction linéaire → O(n²).
096|     Retourne (distances, parents).
097|     """
098|     d      = {x: float("inf") for x in G_pondere}
099|     parent = {}
100|     d[depart] = 0
101|     L = list(G_pondere.keys())          # n = |L| sommets à
traiter
102|
103|     while L:
104|         u = cle_min(d, L)               # extraction du
minimum : O(n)
105|         if u is None:
106|             break
107|         L.remove(u)
108|         for v, w in G_pondere[u]:       # relaxation des
voisins de u
109|             if d[u] + w < d[v]:
110|                 d[v]      = d[u] + w
111|                 parent[v] = u
112|     return d, parent
113|
114|
115| def Reconstruire_Chemin(DictionnaireParents, Depart,
Arrivee):
116|     """Remonte le dictionnaire des parents pour
reconstruire le chemin Depart → Arrivee."""
117|     if Arrivee not in DictionnaireParents and Arrivee !=
Depart:
118|         return None
119|     chemin_inv, ville = [], Arrivee
120|     while ville != Depart:
121|         chemin_inv.append(ville)
122|         if ville not in DictionnaireParents:
123|             return None
124|         ville = DictionnaireParents[ville]
125|     chemin_inv.append(Depart)
126|     return chemin_inv[::-1]
127|
128|
129| def construire_matrice_distances_completes(Graphe, coords,
clients):
```

```
    """
    Matrice n×n des distances routières réelles (plus
courts chemins).
    Appelle dijkstra n fois → O(n³).
    A[i][j] = distance de clients[i] à clients[j].
    """
    G_pondere = creer_graphe_pondere(Graphe, coords)
    n      = len(clients)
    Matrice = np.zeros((n, n))
    for i in range(n):
        dists, _ = dijkstra_vers_tous(G_pondere,
clients[i])
        for j in range(n):
            Matrice[i][j] = dists[clients[j]]
    return Matrice


def creer_matrice_euclidienne_adjacente(Graphe, coords,
clients):
    """
    Matrice n×n des distances euclidiennes directes.
    M[i][j] = dist euclidienne si (i,j) est une arête,
sinon +∞.
    Utilisée pour pondérer les choix locaux des fourmis
(η).
    """
    n = len(clients)
    M = np.full((n, n), np.inf)
    for i, ci in enumerate(clients):
        for j, cj in enumerate(clients):
            if cj in Graphe[ci]:
                M[i][j] = distance_euclidienne(coords[ci],
coords[cj])
    return M


def creer_dict_voisins_indices(Graphe, clients):
    """
    Dictionnaire indice → liste d'indices des voisins
directs.
    Permet un accès O(1) aux voisins pendant la
construction des chemins.
    """
    voisins = {}
    for i, ci in enumerate(clients):
        voisins[i] = [clients.index(cj) for cj in
Graphe[ci]]
    return voisins

```



```

def cout(chemin, n):
    """Nombre de sommets non encore visités = n - |
chemin|."""
    return n - len(chemin)


# ====================================================
# 3. MOTEUR ACO — OUTILS COMMUNS
# ====================================================

def PositionsInitiales(N, start_index=0):
    """Initialise N fourmis au sommet de départ (indice
start_index)."""
    return [[start_index] for _ in range(N)]


def ActualisePheromones(L, A, B, n, N, r, mode):
    """
    Mise à jour de la matrice de phéromones B après une
vague.

    Étape 1 — Évaporation : B[i][j] *= (1 - r)  pour tout
(i,j) → O(n²).
    Étape 2 — Dépôt      : chaque fourmi f dépose 1/S sur
ses arêtes,
                           où S = longueur totale de son
chemin → O(N·n).

    Paramètres
    ----------
    L    : liste des chemins (indices) des N fourmis
    A    : matrice des distances routières n×n
    B    : matrice des phéromones n×n (modifiée en place)
    n    : nombre de villes
    N    : nombre de fourmis
    r    : taux d'évaporation ∈ ]0, 1[
    mode : 1 → circuit fermé (dépôt sur l'arête retour);
autre → chemin ouvert
    """
    # -- Évaporation uniforme --
    for i in range(n):
        for j in range(n):
            B[i][j] *= (1 - r)

    # -- Dépôt proportionnel à 1/longueur --
    for f in range(N):
        chemin = L[f]
        taille = len(chemin)
```

```
        if taille < 2:
            continue

        S = sum(A[chemin[i]][chemin[i + 1]] for i in
range(taille - 1))
        if mode == 1:
            S += A[chemin[-1]][chemin[0]]   # arc de retour
au dépôt

        if S > 0:
            depot = 1.0 / S
            for i in range(taille - 1):
                u, v = chemin[i], chemin[i + 1]
                B[u][v] += depot
                B[v][u] += depot            # graphe non
orienté
            if mode == 1:
                B[chemin[-1]][chemin[0]] += depot
                B[chemin[0]][chemin[-1]] += depot
    return B


# ======================================================
# 4. ACO CLASSIQUE & DYNAMIQUE
# ======================================================

def TourComplet(A, B, M_euc_adj, voisins_indices, n, N,
alpha, beta, gamma,
                depart_fixe=0, mode_dynamique=False):
    """
    Construit les N chemins d'une vague (tournée complète).
    Complexité par vague : O(N · n · k)  avec k = degré
moyen.

    Règle de transition probabiliste (règle ACS) :
        τ(i,j)^α · η(i,j)^β_actuel
    où τ = phéromone, η = 1/distance euclidienne.

    Paramètres
    ----------
    A              : matrice distances routières n×n
    B              : matrice phéromones n×n
    M_euc_adj      : matrice distances euclidiennes (∞ si
non adjacent)
    voisins_indices: dict indice → [indices voisins]
    n              : nombre de villes
    N              : nombre de fourmis
    alpha          : poids des phéromones τ
    beta           : poids de la visibilité η (base)
```



```
    gamma          : intensité de l'adaptation dynamique (0
= classique)
    depart_fixe    : indice du sommet de départ (défaut :
0)
    mode_dynamique : True → ACO Dynamique, False → ACO
Classique
    """
    L = []

    for _ in range(N):                              # --- une
fourmi ---
        chemin       = [depart_fixe]
        non_visites  = list(range(n))
        non_visites.remove(depart_fixe)
        courant      = depart_fixe

        while non_visites:

            # -- β adaptatif (ACO Dynamique) --
            if mode_dynamique:
                c = cout(chemin, n)             # colis
restants à livrer
                if c == 0:
                    c = 0.001                   # garde-fou
division par zéro
                # β croît à mesure que le camion se vide →
comportement de plus en plus glouton
                beta_actuel = beta + gamma * ((n - c) / c)
            else:
                beta_actuel = beta              # β
constant (ACO Classique)

            # -- Calcul des probabilités de transition --
            proba_candidates, proba, denom = [], [], 0

            for j in list(non_visites):
                if j not in voisins_indices[courant]:
                    continue                    # j non
adjacent : ignoré

                dist = M_euc_adj[courant][j]
                if dist == np.inf:              # arête
absente : on retire j
                    non_visites.remove(j)
                    continue
                if dist == 0:
                    dist = 0.001                # évite
division par zéro

```

```
                tau = B[courant][j] ** alpha   # attractivité phéromone
                eta = (1 / dist) ** beta_actuel  # attractivité distance
                val = tau * eta

                proba_candidates.append(j)
                proba.append(val)
                denom += val

            # -- Sélection du prochain sommet --
            if denom == 0:
                # Aucun voisin non visité accessible : repli aléatoire sur un voisin
                voisins_dispo = voisins_indices[courant]
                if voisins_dispo:
                    suivant = random.choice(voisins_dispo)
                else:
                    break
            else:
                # Tirage pondéré par les probabilités normalisées
                proba = np.array(proba) / denom
                suivant = random.choices(proba_candidates, weights=proba)[0]

            chemin.append(suivant)
            if suivant in non_visites:
                non_visites.remove(suivant)
            courant = suivant

        L.append(chemin)
    return L


def CheminLePlusCourt(A, M_euc_adj, voisins_indices, N, NV, r, alpha, beta, gamma,
                      mode_dynamique=False):
    """
    Boucle principale ACO Classique ou Dynamique.
    Complexité totale : O(NV · (N·n·k + n²))  ≈  O(NV · N · n · k).

    À chaque vague :
      1. TourComplet  → N chemins            O(N·n·k)
      2. ActualisePheromones                 O(n² + N·n)
      3. Évaluation des N chemins complets   O(N·n)

    Paramètres
```



```
    ----------
    A              : matrice distances routières n×n
    M_euc_adj      : matrice distances euclidiennes adjacentes n×n
    voisins_indices: dict indice → [indices voisins]
    N              : nombre de fourmis par vague
    NV             : nombre de vagues (itérations)
    r              : taux d'évaporation ∈ ]0, 1[
    alpha          : poids phéromones
    beta           : poids visibilité (base)
    gamma          : intensité adaptation dynamique (0 = classique pur)
    mode_dynamique : True → ACO Dynamique, False → ACO Classique

    Retourne
    --------
    (lmin, meilleur_chemin_noms, historique_distances)
    """
    n                = len(A)
    B                = [[0.01] * n for _ in range(n)]   # phéromones initiales uniformes
    lmin             = np.inf
    best_path_indices = []
    historique       = []

    for _ in range(NV):                                  # --- vague ---
        L = TourComplet(A, B, M_euc_adj, voisins_indices,
                        n, N, alpha, beta, gamma,
                        mode_dynamique=mode_dynamique)
        B = ActualisePheromones(L, A, B, n, N, r, 1)

        for f in range(N):
            chemin = L[f]
            # Chemin valide : visite toutes les villes et retour possible au dépôt
            if len(set(chemin)) < n:
                continue
            if chemin[0] not in voisins_indices.get(chemin[-1], []):
                continue

            S = sum(A[chemin[i]][chemin[i + 1]] for i in range(len(chemin) - 1))
            S += A[chemin[-1]][chemin[0]]           # arc retour au dépôt

            if S < lmin:
```

```
                lmin              = S
                best_path_indices = list(chemin)

        historique.append(lmin)

    if best_path_indices:
        meilleur_chemin = [noms[i] for i in
best_path_indices]
        meilleur_chemin.append(noms[best_path_indices[0]])
# boucle vers le dépôt
    else:
        meilleur_chemin = []

    return lmin, meilleur_chemin, historique


# =====================================================
# 5. ACO HYBRIDE
# =====================================================

def CheminLePlusCourt2(A, G_pondere, M_euc_adj,
voisins_indices, N, NV, r, alpha, beta, gamma):
    """
    Boucle principale ACO Hybride.
    Complexité totale : O(NV · N · n²)  — dominée par les N
Dijkstra par vague.

    Principe : l'aller est construit par ACO (comme le
classique) ;
    le retour optimal est calculé par Dijkstra depuis la
dernière ville.
    Cela permet d'emprunter un chemin retour non contraint
par les phéromones,
    ce qui est avantageux sur les graphes avec culs-de-sac
ou asymétries.

    À chaque vague :
      1. TourComplet  → N chemins aller      O(N·n·k)
      2. ActualisePheromones                 O(n² + N·n)
      3. Pour chaque fourmi valide :
            Dijkstra retour                  O(n²)      ←
surcoût hybride
            Reconstruction chemin            O(n)
      → Surcoût total par vague             O(N·n²)

    Paramètres
    ----------
    A              : matrice distances routières n×n
    G_pondere      : graphe pondéré {u: [(v, dist)]} pour
```



```
Dijkstra
    M_euc_adj      : matrice distances euclidiennes
adjacentes n×n
    voisins_indices: dict indice → [indices voisins]
    N              : nombre de fourmis par vague
    NV             : nombre de vagues
    r              : taux d'évaporation
    alpha          : poids phéromones
    beta           : poids visibilité
    gamma          : non utilisé ici (hybride = classique
sur l'aller)

    Retourne
    --------
    (lmin, aller_noms, retour_noms, historique_distances)
    """
    n                   = len(A)
    B                   = [[0.01] * n for _ in range(n)]
    lmin                = np.inf
    best_aller_indices  = []
    best_retour_indices = []
    historique          = []

    for _ in range(NV):                                  #
--- vague ---
        L = TourComplet(A, B, M_euc_adj, voisins_indices,
                        n, N, alpha, beta, gamma,
                        mode_dynamique=False)
# aller = ACO classique
        B = ActualisePheromones(L, A, B, n, N, r, 1)

        for f in range(N):
            chemin_aller = L[f]

            # Chemin aller valide : toutes les villes
visitées
            if len(set(chemin_aller)) < n:
                continue

            depart  = chemin_aller[0]
            arrivee = chemin_aller[-1]

            dist_aller = sum(A[chemin_aller[i]]
[chemin_aller[i + 1]]
                             for i in
range(len(chemin_aller) - 1))

            # -- Chemin retour optimal par Dijkstra depuis
la dernière ville --
```

```
455|             distances, parents =
dijkstra_vers_tous(G_pondere, noms[arrivee])
456|             chemin_retour_noms =
Reconstruire_Chemin(parents, noms[arrivee], noms[depart])
457|
458|             if not chemin_retour_noms:
459|                 continue
# graphe non connexe : ignoré
460|
461|             chemin_retour = [noms.index(nom) for nom in
chemin_retour_noms]
462|             dist_retour   = distances[noms[depart]]
463|             dist_total    = dist_aller + dist_retour
464|
465|             if dist_total < lmin:
466|                 lmin                = dist_total
467|                 best_aller_indices  = list(chemin_aller)
468|                 best_retour_indices = list(chemin_retour)
469|
470|         historique.append(lmin)
471|
472|     if best_aller_indices:
473|         aller_noms  = [noms[i] for i in best_aller_indices]
474|         retour_noms = [noms[i] for i in
best_retour_indices]
475|     else:
476|         aller_noms = retour_noms = []
477|
478|     return lmin, aller_noms, retour_noms, historique
479|
480|
481| # ====================================================
482| # 6. EXÉCUTION PRINCIPALE
483| # ====================================================
484|
485| def main(mode):
486|     print(f"\n--- ENVIRONNEMENT ACTIF : [{NUMERO_GRAPHE}]
{CATALOGUE_GRAPHES[NUMERO_GRAPHE]['nom']} ---")
487|
488|     # -- Pré-calculs communs aux 3 algorithmes (O(n³)) --
489|     A               =
construire_matrice_distances_completes(Graphe, coords,
LISTE_CLIENTS)
490|     M_euc_adj       =
creer_matrice_euclidienne_adjacente(Graphe, coords,
LISTE_CLIENTS)
491|     voisins_indices = creer_dict_voisins_indices(Graphe,
LISTE_CLIENTS)
492|     G_pondere       = creer_graphe_pondere(Graphe, coords)
```



```
493|
494|     # -- Paramètres ACO communs --
495|     N_FOURMIS  = 20    # nombre de fourmis par vague
496|     NV_VAGUES  = 50    # nombre de vagues (itérations)
497|     R          = 0.5   # taux d'évaporation
498|     ALPHA      = 1     # poids phéromones
499|     BETA       = 1     # poids visibilité (base)
500|
501|     def dessiner_graphe(ax, chemin, titre,
is_hybride=False, aller=None, retour=None):
502|         """Affiche le graphe et le chemin solution sur
l'axe ax."""
503|         # Arêtes du graphe en fond
504|         for u, voisins in Graphe.items():
505|             for v in voisins:
506|                 ax.plot([coords[u][0], coords[v][0]],
507|                         [coords[u][1], coords[v][1]], 'k-',
alpha=0.1)
508|         # Tracé du chemin
509|         if is_hybride and aller and retour:
510|             for k in range(len(aller) - 1):
511|                 ax.plot([coords[aller[k]][0],
coords[aller[k+1]][0]],
512|                         [coords[aller[k]][1],
coords[aller[k+1]][1]], 'g-', linewidth=2, alpha=0.8)
513|             for k in range(len(retour) - 1):
514|                 ax.plot([coords[retour[k]][0],
coords[retour[k+1]][0]],
515|                         [coords[retour[k]][1],
coords[retour[k+1]][1]], 'b--', linewidth=2, alpha=0.8)
516|         elif chemin:
517|             for k in range(len(chemin) - 1):
518|                 ax.plot([coords[chemin[k]][0],
coords[chemin[k+1]][0]],
519|                         [coords[chemin[k]][1],
coords[chemin[k+1]][1]], 'g-', linewidth=2, alpha=0.8)
520|         # Sommets
521|         for sommet, (x, y) in coords.items():
522|             couleur = "orange" if sommet == 'A' else
"white"
523|             ax.scatter(x, y, s=300, c=couleur,
edgecolors="black", zorder=10)
524|             ax.text(x, y, sommet, fontsize=10,
weight="bold", ha='center', va='center', zorder=11)
525|         ax.set_title(titre)
526|         ax.axis("equal")
527|
528|     # ----------------------------------------------------
529|     if mode == 1:
```

```
530|         print("--- Lancement ACO CLASSIQUE ---")
531|         dist, chemin, historique = CheminLePlusCourt(
532|             A, M_euc_adj, voisins_indices, N_FOURMIS,
NV_VAGUES, R, ALPHA, BETA, gamma=0,
533|             mode_dynamique=False)
534|         print(f"Distance : {dist:.2f} | Chemin : {' ->
'.join(chemin)}")
535|         fig, ax = plt.subplots(figsize=(8, 6))
536|         dessiner_graphe(ax, chemin, f"ACO Classique
({dist:.2f})")
537|         plt.show()
538|
539|     elif mode == 2:
540|         print("--- Lancement ACO HYBRIDE ---")
541|         dist, aller, retour, historique =
CheminLePlusCourt2(
542|             A, G_pondere, M_euc_adj, voisins_indices,
N_FOURMIS, NV_VAGUES, R, ALPHA, BETA, gamma=0)
543|         traj_complet = aller[:-1] + retour
544|         print(f"Distance : {dist:.2f} | Chemin complet : {'
-> '.join(traj_complet)}")
545|         fig, ax = plt.subplots(figsize=(8, 6))
546|         dessiner_graphe(ax, None, f"ACO Hybride ({dist:.
2f})", is_hybride=True, aller=aller, retour=retour)
547|         plt.show()
548|
549|     elif mode == 3:
550|         print("--- Lancement ACO DYNAMIQUE (Livreur Pressé)
---")
551|         GAMMA = 100.0   # gamma élevé → comportement très
glouton
552|         dist, chemin, historique = CheminLePlusCourt(
553|             A, M_euc_adj, voisins_indices, N_FOURMIS,
NV_VAGUES, R, ALPHA, BETA, gamma=GAMMA,
554|             mode_dynamique=True)
555|         print(f"Distance : {dist:.2f} | Chemin : {' ->
'.join(chemin)}")
556|         fig, ax = plt.subplots(figsize=(8, 6))
557|         dessiner_graphe(ax, chemin, f"ACO Dynamique
[γ={GAMMA}] ({dist:.2f})")
558|         plt.show()
559|
560|     elif mode == 4:
561|         print("--- COMPARAISON DES 3 STRATÉGIES ---")
562|         GAMMA = 5       # gamma modéré pour la comparaison
563|
564|         dist_c, chem_c, hist_c = CheminLePlusCourt(
565|             A, M_euc_adj, voisins_indices, N_FOURMIS,
NV_VAGUES, R, ALPHA, BETA, gamma=0,
```



```
566|             mode_dynamique=False)
567|         dist_h, all_h, ret_h, hist_h = CheminLePlusCourt2(
568|             A, G_pondere, M_euc_adj, voisins_indices,
N_FOURMIS, NV_VAGUES, R, ALPHA, BETA, gamma=0)
569|         dist_d, chem_d, hist_d = CheminLePlusCourt(
570|             A, M_euc_adj, voisins_indices, N_FOURMIS,
NV_VAGUES, R, ALPHA, BETA, gamma=GAMMA,
571|             mode_dynamique=True)
572|
573|         print(f"Classique : {dist_c:.2f} | Hybride :
{dist_h:.2f} | Dynamique : {dist_d:.2f}")
574|
575|         # -- Carte des 3 solutions --
576|         fig, axs = plt.subplots(1, 3, figsize=(18, 6))
577|         dessiner_graphe(axs[0], chem_c, f"1. Classique
({dist_c:.2f})")
578|         dessiner_graphe(axs[1], chem_d, f"2. Dynamique
γ={GAMMA} ({dist_d:.2f})")
579|         dessiner_graphe(axs[2], None,   f"3. Hybride
({dist_h:.2f})",
580|                          is_hybride=True, aller=all_h,
retour=ret_h)
581|         plt.suptitle(f"Tracés ACO —
{CATALOGUE_GRAPHES[NUMERO_GRAPHE]['nom']}", fontsize=16)
582|         plt.tight_layout()
583|         plt.show()
584|
585|         # -- Courbes de convergence --
586|         vagues_x = range(1, NV_VAGUES + 1)
587|         plt.figure(figsize=(10, 6))
588|         plt.plot(vagues_x, hist_c, label=f"Classique
({dist_c:.2f})",
589|                  color='green', linewidth=2, marker='o',
markersize=4)
590|         plt.plot(vagues_x, hist_d, label=f"Dynamique
γ={GAMMA} ({dist_d:.2f})",
591|                  color='red',   linewidth=2, marker='^',
markersize=4)
592|         plt.plot(vagues_x, hist_h, label=f"Hybride
({dist_h:.2f})",
593|                  color='blue',  linewidth=2, marker='s',
markersize=4)
594|         plt.title(f"Convergence comparée —
{CATALOGUE_GRAPHES[NUMERO_GRAPHE]['nom']}",
595|                   fontsize=14, fontweight='bold')
596|         plt.xlabel("Vagues de fourmis (itérations)",
fontsize=12)
597|         plt.ylabel("Meilleure distance (lmin)",
fontsize=12)
```

```
598|         plt.legend(fontsize=11)
599|         plt.grid(True, linestyle='--', alpha=0.7)
600|         plt.show()
601|
602|
603| if __name__ == '__main__':
604|     print(f"Graphes disponibles (1 à
{len(CATALOGUE_GRAPHES)})")
605|     print("Modes : 1=Classique  2=Hybride  3=Dynamique
4=Comparaison")
606|     mode = int(input("Choix du mode : "))
607|     main(mode)
```